Disturbance-Aware Model-Based and Model-Free Zero-Sum Pursuit-Evasion Control: Theory, Algorithm Development, and MATLAB Validation

A Project Report

Presented to the Faculty of the College of Graduate Studies
Tennessee Technological University

by

Blaine Christopher Swieder

In Partial Fulfillment of the Requirements of the Degree
Master of Science
Electrical and Computer Engineering

August 2026

Abstract

Autonomous and adversarial multi-agent systems require control decisions that remain effective when the inputs applied to a system differ from the commands generated by the controller. This project report investigates disturbance-aware control for a discrete-time, two-player, zero-sum pursuit-evasion game and develops a common framework for extending established model-based and model-free solution methods without replacing their original policy iteration, value iteration, or Q-learning structures. The pursuer is represented as the minimizing player, while the evader is represented as the maximizing player within a linear-quadratic dynamic game. The nominal saddle point policies are characterized through a generalized algebraic Riccati equation and an equivalent quadratic action-value representation.

Four established methods are considered, model-based generalized Riccati policy and value iteration, as well as model-free quadratic Q-learning policy and value iteration. Each method is extended to include persistent command-channel offsets affecting both the pursuer and evader. The offsets represent repeatable differences between commanded and realized inputs. Offset estimates are obtained from transition residuals or measured command mismatch, depending on the information structure of the algorithm. Affine compensation is then applied by subtracting the estimated offsets from the commanded control actions. As a result, the resulting residual-bias dynamics directly relate the compensated closed-loop response to the remaining estimation error and establish exact compensation as a verification case.

MATLAB studies evaluate nominal operation, uncompensated command offsets, estimated-offset compensation, and exact compensation under matched system matrices, game weights, disturbances, initial conditions, and numerical criteria. The validation includes policy and value convergence, generalized Riccati and Bellman residuals, closed-loop spectral radius, saddle-point curvature, matrix conditioning, feature-rank requirements, bias-estimation error, state and input trajectories, and cumulative game cost. This project report does not introduce new base algorithms; moreover, its contribution is the unified disturbance-aware extension and matched evaluation of four established zero-sum control methods within a common pursuit-evasion framework. Pursuit-evasion provides the application interpretation, while nonlinear autonomous ground vehicle and QCar implementations are retained as future extensions rather than completed hardware validation.

Contents

Abstract i

1 Introduction and Literature Review 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Research Questions and Objectives . . . 4
- 1.4 Zero-Sum Linear-Quadratic Dynamic Games and the GARE . . . 5
- 1.5 Model-Based Policy and Value Iteration . . . 7
- 1.6 Model-Free Quadratic Q-Learning and Adaptive Dynamic Programming . . . 8
- 1.7 Persistent Command Bias, Unknown-Input Estimation, and Affine Compensation . . . 10

2 Problem Formulation and Nominal Riccati Solution 13
- 2.1 Role of the Unified Framework . . . 13
- 2.2 Discrete-Time Zero-Sum Pursuit-Evasion Dynamics . . . 14
- 2.3 Pursuer, Evader, and State-Space Definitions . . . 15
- 2.4 Quadratic Game Objective and Saddle-Point Policies . . . 15
- 2.5 Equivalent Nominal Value Representations . . . 17
- 2.6 Nominal GARE Policy Iteration . . . 18
- 2.7 Nominal GARE Value Iteration . . . 20

3 Model-Based Bias Compensation: Policy and Value Iteration 22
- 3.1 Persistent Command-Channel Offset Model . . . 22
- 3.2 Model-Based Transition Residual Estimation . . . 23
- 3.3 Affine Compensation and Residual Closed-Loop Behavior . . . 24
- 3.4 Disturbance-Aware Extension of GARE Policy Iteration . . . 25
- 3.5 Disturbance-Aware Extension of GARE Value Iteration . . . 29
- 3.6 Model-Based GARE Policy Iteration Results . . . 30
- 3.7 Model-Based GARE Value Iteration Results . . . 33

4 Model-Free Q-Learning Formulation, Algorithms, and Results 36
- 4.1 Model-Free Information Structure and Common Action-Value Objects . . . 36
- 4.2 Nominal Q-Learning Policy Iteration . . . 39
- 4.3 Disturbance-Aware Extension of Q-Learning Policy Iteration . . . 40
- 4.4 Nominal Q-Learning Value Iteration . . . 42
- 4.5 Disturbance-Aware Extension of Q-Learning Value Iteration . . . 43
- 4.6 Model-Free Q-Learning Policy Iteration Results . . . 44

4.7 Model-Free Q-Learning Value Iteration Results 47
4.8 Matched Four-Method Comparison 48

5 Conclusions and Future Work **50**
5.1 Summary 50
5.2 Further Research 51

References **53**

Chapter 1

Introduction and Literature Review

1.1 Background and Motivation

Autonomous and cyberphysical systems are increasingly operating in environments where their performance relies on the decisions of another agent. As a result, in these settings, the control problem is not only to stabilize a plant or track a prescribed reference; rather, the controller must act while another decision maker aims to influence the same state in an opposing direction. Pursuit-Evasion Games provide one of the clearest mathematical representations of this interaction. More specifically, a pursuer attempts to reduce a relative error or drive the state toward a capture condition, while an evader attempts to increase the same cost or prevent the capture condition from being reached. As a result, the pursuit-evasion problem can convert an open-ended adversarial interaction into a controlled two-player game whose state, inputs, objective, and equilibrium conditions are explicitly defined [1, 2].

The linear-quadratic specialization is important in this context since it retains the strategic structure of the game while allowing the equilibrium policies to be computed from matrix equations. To be more precise, a discrete-time linear system with quadratic state and input penalties leads to a two-player zero-sum game in which one player minimizes the performance index while the other maximizes it. The corresponding saddle-point solution is connected to a Game Algebraic Riccati Equation or GARE, and the resulting policies are linear functions of the state. Therefore, this formulation is essential as it gives a transparent model-based baseline whose assumptions, numerical conditions, and closed-loop behavior are able to be examined more directly [1, 3].

On the contrary, the model-based solution is only one side of the control problem. A GARE-based controller requires the system matrices to be known, while many practical systems provide measured input-output or state-transition data without providing an exact analytical model. Model-free quadratic Q-learning addresses this difference by estimating an action-value function from sampled data and recovering the same game policies from the estimated quadratic matrix. Some of the early literature demonstrated that a linear discrete-time zero-sum game could be solved by utilizing model-free Q-learning and that the resulting iterations converge to the nominal Riccati solution under the required excitation and rank conditions [4, 5]. Later work in the literature demonstrates that policy iteration Q-learning methods further develop this idea in the context of data-based two-player zero-sum systems [6].

This distinction between model-based and model-free should not be interpreted as a comparison between an analytical method and an unrelated learning method. Both approaches do solve the

same zero-sum game, use the same quadratic objective, and seek the same pursuer and evader policies. Where these two methods differ is the information that is used during the computation. The model-based methods evaluate or update the value matrix using the known system dynamics, whereas the model-free methods reconstruct the required quadratic relationships from measured transitions. Rizvi et al. organize these ideas through corresponding policy iteration and value iteration formulations for nominal linear quadratic zero-sum games [5]. As a result, this shared structure creates a direct foundation for studying four algorithms under the same problem definition; that is, model-based policy iteration, model-based value iteration, model-free Q-learning policy iteration, and model-free Q-learning value iteration.

In addition, more recent literature aims to refine each part of this framework. Recent model-based literature has developed single-loop policy iteration methods for linear zero-sum games with relaxed initialization requirements [7]. Similarly, recent value iteration research has also established on-policy and off-policy formulations for stochastic zero-sum linear quadratic games with unknown dynamics [8]. Modern learning theory has investigated convergence, last-iterate behavior, and sample complexity for zero-sum linear quadratic games [9]. Therefore, these developments within the literature demonstrate that the linear quadratic zero-sum game is an active area within the current literature rather than solely a historical control benchmark.

Nevertheless, much of the nominal algorithmic literature assumes that the input computed by the controller is the input that enters the plant. This assumption is convenient for the derivation; however, it excludes a common execution-side problem. That is, a persistent difference between a commanded and realized input may represent an actuator calibration offset, steering-center shift, longitudinal-command mismatch, sensor-to-command conversion error, or another repeatable implementation bias. Even when the nominal feedback gains stabilize the game, a persistent input offset introduces an affine forcing term into the closed-loop dynamics and may create a nonzero deviation from the nominal trajectory.

This is a critical distinction since nominal convergence of the value matrix does not imply that the realized system will follow the nominal closed-loop response. A policy may satisfy the GARE, the saddle-point curvature conditions, and the closed-loop stability condition while the implemented trajectory still settles away from its nominal equilibrium. In pursuit-evasion research, the current literature has similarly moved towards safety, disturbance rejection, sensor limitations, and physical implementation [10, 11, 12].

Therefore, this project report direction is intended to preserve the established four-algorithm zero-sum framework and extend it with a common persistent command-offset model, bias-estimation procedure, affine compensation law, and MATLAB validation structure. The contribution is not intended to be a novel invention of the underlying policy iteration, value iteration, or Q-learning equations. Instead, the contribution is the systematic development of disturbance-aware versions that remain traceable to the original nominal algorithms while allowing the effect of imperfect command realization to be measured and compensated.

1.2 Problem Statement

This project report considers a discrete-time two-player zero-sum system in which the pursuer acts through one input channel and the evader acts through another. Thus, the nominal game dynamics can be given as follows

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k},$$

where $x_k \in \mathbb{R}^n$ denotes the game state, $u_{P,k} \in \mathbb{R}^{m_P}$ denotes the pursuer input, $u_{E,k} \in \mathbb{R}^{m_E}$ denotes the evader input, and A, B_P and B_E are the state and input matrices. The pursuer is the minimizing player, while the evader is the maximizing player.

A representative infinite-horizon quadratic objective is expressed as

$$J = \sum_{k=0}^{\infty} (x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k}),$$

where $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. Under the nominal saddle-point policies, the two inputs can be written as

$$u_{P,k} = Lx_k, \quad u_{E,k} = Kx_k,$$

where L and K are the pursuer and evader feedback gains. The corresponding closed-loop matrix is $A + B_P L + B_E K$.

The nominal model-based methods obtain the value matrix and the two gains from GARE-based policy iteration or value iteration updates. The nominal model-free methods estimate a quadratic Q-function from measured data and recover the same policies from the partitions of the estimated action-value matrix. In the algorithms that are used as a starting point for this project report, these two pairs of methods have different information requirements, but they are directed toward the same saddle-point solution [5].

The practicality that is considered here is that the realized player inputs may differ from their commanded values. The command-channel model is given by

$$u_{P,k}^{\text{act}} = u_{P,k}^{\text{cmd}} + b_P, \quad u_{E,k}^{\text{act}} = u_{E,k}^{\text{cmd}} + b_E$$

where b_P and b_E denote constant or slowly varying offsets over the calibration interval and the following closed-loop run. When no compensation is applied, the realized dynamics then become,

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P b_P + B_E b_E.$$

As a result, the nominal gains may remain stabilizing while the affine offset changes the steady-state response, the state trajectory, and the accumulated game cost.

The command offsets are estimated according to the information available to each algorithmic class. For the model-based methods, the known system matrices allow a transition residual to be formed from measured states and commanded inputs. For the model-free methods, the offset estimate is obtained from measured command-channel data or from an augmented data regression that does not require the analytical state-transition matrices during the Q-learning update.

Then, the compensated commands are expressed as follows

$$u_{P,k}^{\text{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\text{cmd}} = Kx_k - \hat{b}_E.$$

As a result, the compensated closed-loop system then becomes

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P(b_P - \hat{b}_P) + B_E(b_E - \hat{b}_E).$$

This equation is crucial as it provides the central foundation of this project report. If the offset estimates are exact, then the nominal closed-loop dynamics are recovered. On the other hand, if the estimates are imperfect, the remaining affine term relies only on the estimation errors rather than on the full command offsets.

The research problem here is consequently not to replace the original GARE or Q-learning algorithms. Instead, it is to determine whether the four established methods are able to be extended under a common disturbance model while their original nominal iteration structures remain identifiable. The extended implementations must also be evaluated using common numerical criteria so that differences caused by the information structure are not confused with differences caused by inconsistent simulation conditions.

1.3 Research Questions and Objectives

There are five key research questions in this project report that connect the nominal zero-sum solution, the command-offset model, the estimation process, the compensation law, and the MATLAB comparison. These five research questions are as follows:

1. *Nominal Algorithm Reproduction*: Can the model-based policy iteration, model-based value iteration, model-free Q-learning policy iteration, and model-free Q-learning value iteration methods be implemented such that their nominal equations remain traceable to the established zero-sum algorithms from which they are derived?

2. *Persistent Command-Offset Effect*: How do constant pursuer and evader command offsets alter the nominal state response, predicted steady-state deviation, finite-window game cost, and closed-loop trajectory when the feedback gains themselves are unchanged?

3. *Bias Estimation and Compensation*: Can the persistent command offsets be estimated using the information available to the model-based and model-free methods, and can affine input compensation reduce the resulting deviation from nominal behavior?

4. *Matched Four Algorithm Comparison*: How do policy iteration and value iteration compare within the model-based and model-free classes when all four methods use the same game matrices, disturbance definitions, calibration conditions, simulation horizon, and numerical diagnostics?

5. *Pursuit-Evasion and Future Research Directions*: To what extent does the resulting disturbance-aware framework provide a defensible mathematical and computational foundation for later pursuit-evasion studies that involve nonlinear vehicles, QLabs, or physical QCar platforms?

From these research questions, the following five primary research objectives are devised.

- *Objective 1*: Reproduce and document the nominal model-based and model-free policy and value iteration methods without presenting the underlying algorithms as new contributions.

- *Objective 2*: Introduce a common persistent command channel offset model for the pursuer and evader and derive its effect on the realized closed-loop dynamics.

- *Objective 3*: Develop information consistent offset-estimation and affine-compensation procedures for the model-based and model-free implementations.
- *Objective 4*: Establish a shared MATLAB validation protocol using convergence, GARE residual, closed-loop spectral radius, saddle-point curvature, regression rank, conditioning, bias-estimation error, trajectory deviation, steady-state offset, and finite-window cost.
- *Objective 5*: Interpret the resulting zero-sum system as a pursuit-evasion benchmark and identify the precise work that remains before nonlinear vehicle or hardware claims are able to be made.

1.4 Zero-Sum Linear-Quadratic Dynamic Games and the GARE

Differential Game Theory formalizes decision problems in which several players influence the same dynamical system while pursuing different objectives. Isaacs established the classical pursuit-evasion and differential game framework, including the interpretation of opposing players and terminal capture conditions [13]. Later, Basar et al. developed the general theory of dynamic noncooperative games, including zero-sum, Nash, open-loop, and feedback formulations [1]. These foundational works are crucial since the language that is used in this project report, such as the player, policy, value, saddle-point, and zero-sum objective, all originate from this game-theoretic structure.

The zero-sum case is distinguished by a single performance index that one player attempts to minimize and the other attempts to maximize. For a discrete-time linear system, the nominal dynamics can be represented by the equation

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k}, \tag{1.1}$$

where x_k denotes the game state, $u_{P,k}$ denotes the minimizing-player input, and $u_{E,k}$ denotes the maximizing-player input. A representative infinite-horizon quadratic cost can be defined by the equation

$$J = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k} \right), \tag{1.2}$$

where $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. The sign difference between the two input penalties reflects the minimax structure. The pursuer attempts to reduce the state and its own control effort, while the evader acts to increase the same game value within the admissible saddle conditions.

The linear-quadratic structure is crucial, since it produces policies that are linear in the state and a value function that is quadratic. More specifically, if a stabilizing saddle solution exists, then the value can be expressed as

$$V(x_k) = x_k^\top P x_k, \tag{1.3}$$

and the policies that take the form of

$$u_{P,k} = Lx_k, \quad u_{E,k} = Kx_k. \tag{1.4}$$

The resulting closed-loop matrix is $A+B_PL+B_EK$. Pachter et al. provide a discrete-time treatment of linear-quadratic dynamic games that is especially relevant because the MATLAB algorithms contained in this project report operate in sampled time as opposed to through continuous-time differential equations [3]. The discrete-time formulation also makes the relationship between the Bellman equation, the GARE, and quadratic Q-learning more apparent.

The Game Algebraic Riccati Equation, or GARE, is the central matrix equation for the nominal model-based problem. Moreover, its block structure contains the interactions among the state, minimizing input, and maximizing input. Unlike the single-player LQR problem, the zero-sum game needs to satisfy both a minimizing-player curvature condition and a maximizing-player curvature condition. The minimizing-player control block must be positive definite, while the appropriate maximizing-player Schur complement must be negative definite. These conditions are not secondary numerical conveniences; rather, they determine whether the stationary point obtained from the joint quadratic form is a saddle point instead of a minimum in both inputs or an indefinite stationary point.

The pursuit-evasion specialization of linear-quadratic games has a long history. Ho established an early analytical result for LQ pursuit-evasion differential games, while Bagchi et al. extended the setting to stochastic disturbances [14, 15]. Shinar examined solvability conditions for LQ differential games associated with pursuit-evasion problems, demonstrating that an apparently reasonable quadratic formulation does not automatically guarantee the existence of an admissible saddle solution [16]. These results support the decision in this project report to report curvature and stability checks explicitly rather than infer validity from the existence of a numerical matrix alone.

In addition, more recent research continues to study the distinction between open-loop and feedback equilibria and the behavior of zero-sum LQ games under stochastic or long-horizon conditions. Sun et al. distinguish open-loop and closed-loop saddle points in stochastic LQ differential games, while Sun later develops the two-person stochastic zero-sum case in greater detail [17, 18]. Monti et al. examine feedback and open-loop Nash equilibria for infinite-horizon discrete-time dynamic games, which reinforces the importance of stating which equilibrium concept is implemented by an algorithm [19]. Sun and Yong's more recent long-time analysis further demonstrates that the LQ game remains an active theoretical setting rather than only a historical benchmark [20].

The current learning literature also treats zero-sum LQ games as a demanding benchmark. Wu et al. emphasize that the corresponding optimization problem is nonconvex and nonconcave and develop improved sample-complexity and last-iterate convergence results [9]. This work is crucial for this project report since it places the model-free algorithms within the current state of research while also clarifying that convergence claims depend on the precise algorithm, information assumptions, and admissible policy region. This project report does not aim to reproduce that sample-complexity theory. Instead, it uses the established LQ game structure to compare the four algorithms under a shared persistent command-offset model.

Therefore, the Game Algebraic Riccati Equation (GARE) literature establishes three requirements for the remainder of the project report. First, the nominal problem must be stated as a genuine minimax game rather than two independent controllers. Second, the feedback gains must be associated with a stabilizing value matrix and valid saddle curvature. Third, any disturbance-aware extension should preserve the nominal GARE structure if the project report claims to extend rather than replace the original algorithms. These requirements provide the basis for the model-based policy iteration and model-based value iteration literature that is in the following section.

1.5 Model-Based Policy and Value Iteration

Policy Iteration and Value Iteration are related dynamic programming procedures, but they should not be treated as interchangeable names for the same computation. Policy iteration alternates between evaluating fixed policies and improving those policies, while value iteration repeatedly updates the value function through a Bellman or Riccati mapping and then obtains the policies from the current value estimate. The two procedures may converge to the same nominal solution under compatible assumptions, but their intermediate matrices, initialization requirements, and numerical behavior may differ.

The policy iteration lineage is connected to iterative Riccati methods developed for optimal regulation. Kleinman introduced an iterative Riccati technique that is based on repeated Lyapunov equations, and Hewer developed a discrete-time steady-state gain iteration [21, 22]. Although these studies concern single-player regulation, they establish the basic computational principle that a nonlinear Riccati equation is able to be approached through a sequence of linear matrix equations associated with current policies. Mageirou extended iterative ideas to Riccati game equations, making the link to dynamic games explicit [23].

For a zero-sum game, policy evaluation is performed with the current minimizing and maximizing gains fixed. If L_i and K_i are the policies at iteration i, then the current closed-loop matrix is given by

$$A_i = A + B_P L_i + B_E K_i. \tag{1.5}$$

Provided that A_i is Schur stable, the associated value matrix can be obtained from a discrete Lyapunov equation. Policy improvement then updates both gains from the blocks of the game quadratic form. The two improvements are coupled because the pursuer gain relies on the evader blocks and the evader gain depends on the pursuer blocks. As a result, this coupling is one reason that a zero-sum policy iteration cannot be reduced to two independent LQR updates.

Abu-Khalaf et al. developed policy iterations for the Hamilton-Jacobi-Isaacs equation in H_∞ state-feedback control with input saturation, demonstrating how the game interpretation connects disturbance attenuation and minimax control [24]. Wu et al. later proposed simultaneous policy updates for learning continuous-time H_∞ control solutions [25]. These studies are relevant since the pursuer-vs-evader interpretation and the controller-versus-disturbance interpretation share the same zero-sum mathematical structure, even when the physical meanings of the two players differ.

Rizvi et al. organize model-based policy iteration and value iteration alongside their model-free counterparts for linear systems [5]. As a result, this organization is the direct starting point for the algorithmic structure in this project report. The model-based policy iteration code sourced from the author uses known matrices to evaluate a closed-loop policy and then improves the two gains. The model-based value iteration source code repeatedly applies the GARE mapping and then computes the policies associated with the updated value matrix. The present disturbance-aware work retains these roles. The command-offset estimation and compensation procedures are added to the execution and evaluation layers rather than inserted into the nominal Riccati equations without justification.

In addition, recent research has continued to refine zero-sum policy iteration. Zhao et al. propose a single-loop policy iteration method for linear zero-sum games and establish convergence for the resulting GARE sequence [7]. The importance of this study is not that every earlier policy iteration is obsolete; rather, it demonstrates that initialization, loop structure, and convergence proof

remain active concerns. Similarly, Li et al. develop a relaxed policy iteration algorithm for nonlinear zero-sum games with application to H_∞ control [26]. Nortmann et al. examine iterative and data-driven algorithms for linear-quadratic discrete-time dynamic games, further demonstrating the current interest in linking model-based game solutions to data-driven computation [27].

On the other hand, value iteration follows different computational logic. Starting from an initial value matrix, the algorithm repeatedly applies a Riccati or Bellman operator. The policies are functions of the current value matrix rather than fixed during a separate evaluation step. Bertsekas provides the broader dynamic programming foundation for this distinction [28]. In zero-sum games, the value update must preserve the minimax block structure and remain within a region where the necessary matrix inverses or linear solvers are well-defined.

Guo et al. formulate on-policy and off-policy value iteration algorithms for stochastic zero-sum dynamic games with unknown dynamics [8]. Their research is crucial, since it treats both model-based convergence and model-free forms within one value iteration setting. Moreover, it also shows that the line between model-based and model-free computation is determined by the information used in the update, not simply by whether the word *value iteration* appears in the algorithm title.

Therefore, the literature suggests several principles for this project report. First, policy iteration and value iteration should be implemented and reported separately. Second, changes that are made for numerical robustness, such as replacing explicit matrix inversion with a linear solve, should be identified as implementation choices rather than novel control laws. Third, the original update equations should remain recognizable when the disturbance-aware layer is added. Fourth, convergence histories should report changes in the value matrix and both gains since a small value update does not automatically prove that all policy quantities have converged.

These principles also clarify the role of the model-based algorithms in the final comparison. They are not used merely to generate a single reference matrix, and they provide two established procedures with different iteration structures but the same nominal system knowledge. The disturbance-aware study then asks whether the same command-offset model and compensation logic can be applied without altering what each nominal algorithm does.

1.6 Model-Free Quadratic Q-Learning and Adaptive Dynamic Programming

Model-free control is often discussed as if it only refers to deep neural network policies trained through long episodes; however, that framing is too broad for this project report. The model-free methods that are considered here are structured quadratic Q-learning algorithms for linear zero-sum systems. The purpose of these model-free methods is to recover the value and policy information that is needed by the game without inserting the analytical state-transition matrices into the learning update.

Watkins et al. established Q-learning as a model-free temporal difference method for estimating optimal action values [29]. In addition, Sutton and Barto provide the broader reinforcement learning framework, including value functions, Bellman equations, exploration, and temporal difference learning [30]. For linear-quadratic control, however, the action-value function has additional structure. Bradtke et al. developed linear least-squares temporal difference methods, and then Lagoudakis et al. later developed least-squares policy iteration [31, 32]. These studies establish the general connection between sampled transitions, linear parameter estimation, and policy improvement.

In a two-player zero-sum linear-quadratic game, the joint state-action vector can be written as

$$\zeta_k = \begin{bmatrix} x_k^\top & u_{P,k}^\top & u_{E,k}^\top \end{bmatrix}^\top, \tag{1.6}$$

and the quadratic action-value function is represented by

$$Q_H(\zeta_k) = \zeta_k^\top H \zeta_k, \tag{1.7}$$

where H is said to be symmetric. The state, pursuer-input, evader-input, and cross-coupling blocks of H contain the information needed to recover the two saddle policies. Thus, the learner estimates a finite set of quadratic coefficients rather than an unrestricted nonlinear function.

Al-Tamimi et al. provide foundational adaptive critic and model-free Q-learning designs for discrete-time zero-sum games with application to H_∞ control [4, 33]. Their work is important since it demonstrates that the GARE solution can be recovered from data through a quadratic Q-function without direct knowledge of the system matrices. Additionally, Lewis et al. place these ideas within adaptive dynamic programming and feedback control more generally [34]. These sources form the conceptual link from nominal Riccati computation to the model-free algorithms in this project report.

Policy iteration Q-learning evaluates the Q-function associated with current pursuer and evader policies and then improves the policies from the estimated matrix partitions. Luo et al. develop a policy iteration Q-learning method for data-based two-player zero-sum linear discrete-time systems [6]. The method emphasizes that the data matrix must contain sufficient independent quadratic information. In practical terms, the samples must excite the state and both input channels strongly enough that the symmetric parameters of H can be identified.

Finite-horizon and output-feedback extensions broaden this structured learning framework. Liu et al. develop finite-horizon Q-learning for discrete-time zero-sum games and later output-feedback Q-learning for finite-horizon problems [35, 36]. These are relevant studies since they show that the quadratic zero-sum learning structure is not limited to one infinite-horizon full-state formulation. Lian et al. consider off-policy inverse Q-learning for unknown antagonistic systems, while Guo et al. develop Q-learning for stochastic zero-sum games [37, 38].

Recent studies extend similar ideas toward nonlinear, switched, delayed, and safety-constrained systems. Wang et al. propose neural Q-learning for discrete-time nonlinear zero-sum games with an adjustable convergence rate [39]. Additionally, Wang et al. also develop a parallel off-policy Q-learning method for discrete-time Markov jump systems [40]. Jiang et al. address delayed two-player zero-sum games through data-based Q-learning, and Liu et al. study safe reinforcement learning for nonlinear zero-sum games with unknown state constraints and asymmetric input constraints [41, 42]. These papers represent recent developments, but they also help to define the boundary of the present work. The project report does not introduce neural approximators, Markov jumps, unknown safety sets, or delayed dynamics into the four baseline algorithms. Instead, it develops a controlled persistent-offset extension whose effect can be isolated.

The literature also distinguishes structured zero-sum Q-learning from policy gradient optimization. Zhang et al. establish convergence results for policy optimization in zero-sum LQ games, while Wu et al. later improve sample-complexity and last-iterate guarantees [9, 43]. These papers are essential state-of-the-art references; however, they do not replace the quadratic Q-learning algorithms used as the project report starting point. In addition, they demonstrate that zero-sum LQ learning can be approached through different computational routes, each with different theoretical

assumptions.

There are several numerical requirements that follow directly from the model-free literature. The feature matrix needs to have sufficient rank to identify the symmetric Q-function parameters. The regression matrix must remain adequately conditioned; moreover, the behavior policies need to provide excitation without driving the system outside the region in which the data are meaningful. The estimated game matrix must satisfy the same minimizing and maximizing curvature requirements used by the model-based solution. Finally, a small Bellman residual should be interpreted together with gain error, rank, conditioning, and closed-loop behavior rather than used as a stand-alone proof of policy recovery.

The present project report uses these principles to keep the model-free comparison disciplined. The model-free label refers to the absence of the analytical transition matrices from the Q-learning update. It does not imply that no measurements, no prior structural assumptions, or no commanded-versus-realized input data are available. This is an important distinction when persistent command offsets are introduced, since the policy may be learned without the matrices while the bias estimate may still use measured actuator-channel information.

1.7 Persistent Command Bias, Unknown-Input Estimation, and Affine Compensation

The disturbance-aware contribution starts with a simple but consequential observation. That is, a control command and a realized control input are not always identical. A persistent mismatch may be caused by actuator calibration, steering-center error, longitudinal response offset, conversion scaling, or another repeatable implementation effect. If the mismatch remains approximately constant over a calibration batch and the following run, then it can be represented as an additive command-channel bias.

For the two-player game, the realized inputs are modeled as

$$u_{P,k}^{\mathrm{act}} = u_{P,k}^{\mathrm{cmd}} + b_P, \quad u_{E,k}^{\mathrm{act}} = u_{E,k}^{\mathrm{cmd}} + b_E. \tag{1.8}$$

When the nominal policies are applied without compensation, the closed-loop dynamics become

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P b_P + B_E b_E. \tag{1.9}$$

The additional term is affine and persistent. If the nominal closed-loop matrix is Schur stable, then the state need not diverge. Instead, it may approach a nonzero equilibrium determined by the bias and the closed-loop resolvent. This behavior is essential since a nominal stability result can coexist with a meaningful trajectory error.

The offset-free control literature provides a mature foundation for representing this problem. Muske et al. develop disturbance modeling for offset-free linear model predictive control, while Pannochia et al. develop disturbance models that remove steady-state tracking offset [44, 45]. Maeder et al. provide a systematic treatment of linear offset-free MPC, and Jimoh et al. compare different offset-free formulations [46, 47]. Although these studies are primarily written in the MPC setting, the underlying principle is directly relevant. More specifically, persistent mismatch should be represented explicitly rather than treated as random numerical noise.

The unknown-input estimation literature addresses the related problem of reconstructing unmeasured forcing terms. Gillijns et al. derive unbiased minimum-variance input and state estima-

tors for linear discrete-time systems [48]. Later, Yong et al. provide a unified filter for simultaneous input and state estimation [49]. These methods are more general than the batch constant-bias estimator that is used in the present model-based MATLAB benchmark. Nevertheless, they establish the identifiability and estimation principles that justify treating a persistent command offset as an estimable input rather than an arbitrary unexplained deviation.

For the known-model case, the transition residual is expressed by

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}. \tag{1.10}$$

Under the bias model, this residual satisfies

$$r_k = \begin{bmatrix} B_P & B_E \end{bmatrix} \begin{bmatrix} b_P \\ b_E \end{bmatrix} + \eta_k, \tag{1.11}$$

where η_k denotes measurement noise, process noise, and finite-precision effects. A batch least-squares estimate is possible when the combined bias map has sufficient rank. This requirement is physically meaningful. If the two input channels affect the state in indistinguishable directions, the individual player biases cannot be separated from state-transition alone.

Once the offsets are estimated, affine command compensation subtracts the estimates from the nominal policy outputs:

$$u_{P,k}^{\mathrm{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\mathrm{cmd}} = Kx_k - \hat{b}_E. \tag{1.12}$$

The realized dynamics then become

$$x_{k+1} = (A + B_P L + B_E K)x_k + B_P(b_P - \hat{b}_P) + B_E(b_E - \hat{b}_E). \tag{1.13}$$

Exact estimates recover the nominal closed-loop dynamics, while imperfect estimates leave only the residual bias. This relationship provides a direct interpretation of the numerical results. Compensation performance should be consistent with the remaining estimation error rather than described as an unexplained improvement.

Recent studies continue to research identification and offset-free stability under more general conditions. Berberich et al. develop data-driven tracking MPC for nonlinear systems, showing how steady-state tracking and model-free data can be connected [50]. Kuntz and Rawlings examine maximum-likelihood identification with integrating disturbances and offset-free stability under plant-model mismatch [51, 52]. Yang et al. consider joint state and bias estimation for nonlinear systems under non-Gaussian process noise [53]. These studies extend beyond the constant-bias benchmark; however, they demonstrate that bias estimation and offset-free control remain active research topics.

In addition, the literature also clarifies what is and is not claimed by the present compensation method. The estimator is not a universal unknown-input observer, and it does not guarantee identification under arbitrary time-varying disturbances. The compensation law does not redesign the saddle policy nor solve a new robust game. Rather, it assumes that the nominal policy has already been obtained and that the persistent command mismatch can be estimated over a calibration interval. Thus, the resulting affine correction is best described as a disturbance-aware execution layer attached to an established nominal algorithm.

This distinction is crucial to the project report contribution. The policy iteration, value iteration, and Q-learning equations retain their original mathematical roles. The added work defines

how the commanded policies are translated into realized inputs, how persistent offsets are estimated under the information available to each method, and how compensation quality is measured through nominal, uncompensated, estimated-compensation, and exact-compensation cases.

Chapter 2

Problem Formulation and Nominal Riccati Solution

2.1 Role of the Unified Framework

The prior chapter established that the literature supports zero-sum dynamic games, model-based Riccati methods, model-free quadratic Q-learning, persistent input estimation, and affine compensation. The objective of this chapter is to define the common mathematical problem that is used by all four algorithms in this project report. More specifically, this chapter establishes the state equation, the minimizing and maximizing inputs, the quadratic game objective, the saddle-point policies, the Game Algebraic Riccati Equation (GARE), the quadratic action-value function, the persistent command-offset model, and the numerical conditions that are required before a computed solution can be interpreted as a valid zero-sum controller.

This is a necessary common formulation, since the model-based and model-free methods are not intended to solve different control problems. Model-based policy and value iteration, as well as model-free Q-learning policy and value iteration, all refer to the same discrete-time two-player game. Their principal distinction is the information that is used during the computation. The model-based methods evaluate matrix equations by using the known system matrices, while the model-free methods estimate the corresponding quadratic relationships from sampled state and input data. Therefore, a meaningful comparison requires the plant, cost, player interpretation, command offsets, compensation cases, and validation metrics to remain fixed across the four methods [5, 4].

In addition, the chapter also clarifies what the current pursuit-evasion interpretation does or does not claim. The present MATLAB benchmark is a reduced-order linear zero-sum system inherited from the source algorithm files [5]. It is interpreted as a local pursuit-evasion engagement in which the minimizing player acts as the pursuer and the maximizing player acts as the evader. However, the three-state benchmark is not presented as a complete nonlinear vehicle model that contains planar position, heading angle, wheelbase, acceleration, and steering. Those physical states remain part of the future vehicle extension. In this project report, pursuit-evasion provides the game interpretation, while the controlled linear system provides the mathematical foundational benchmark on which the disturbance-aware algorithm extensions are developed.

2.2 Discrete-Time Zero-Sum Pursuit-Evasion Dynamics

The nominal system is a discrete-time linear plant with one minimizing input channel and one maximizing input channel, and can be expressed by the following equation

$$x_{k+1} = Ax_k + B_P u_{P,k} + B_E u_{E,k}, \tag{2.1}$$

where $x_k \in \mathbb{R}^n$ denotes the game state at time step k, $u_{P,k} \in \mathbb{R}^{m_P}$ denotes the pursuer input, and $u_{E,k} \in \mathbb{R}^{m_E}$ denotes the evader input. The matrices $A \in \mathbb{R}^{n\times n}$, $B_P \in \mathbb{R}^{n\times m_P}$, and $B_E \in \mathbb{R}^{n\times m_E}$ define the nominal state and player-input channels. From the MATLAB sources, B_P is denoted by B, while B_E is denoted by E. The notation used in this chapter aims to make the pursuer and evader roles explicit.

Notice that the state equation in Equation (2.1) is shared by both players. As a result, this means that the pursuer and evader do not operate separate plants. Rather, their inputs act on the same state, and the resulting state transition reflects the combined effect of both policies. The pursuer attempts to regulate the state in the direction favored by the performance objective, while the evader attempts to oppose that regulation. This coupled structure is what distinguishes the problem from two independent optimal control problems [1, 3].

A state-space formulation is useful for three reasons. First, it makes the effect of each player explicit through B_P and B_E. Second, it supports a quadratic value function and a GARE-based saddle solution. Third, it allows the same transition data to be reorganized into a quadratic Q-learning regression. Therefore, the common state-space model provides the analytical baseline for the model-based methods and the data-generating environment for the model-free methods.

The present benchmark is time-invariant, fully-observed, and deterministic during the nominal control calculation. These assumptions are intended to be restrictive as time variation, partial state measurement, stochastic process noise, actuator saturation, and nonlinear dynamics are important extensions. However, including these extensions at this stage would make it difficult to determine whether an observed difference is caused by the original algorithm, the command-offset extension, or an unrelated modeling change. As a result, the current formulation starts with the smallest system that is able to preserve the zero-sum saddle structure and the distinction between known-model and data-based computation.

Table 2.1: Common Symbols used in the Disturbance-Aware Zero-Sum Formulation

Symbol	Meaning
x_k	Shared game state at time step k.
$u_{P,k}$	Pursuer or minimizing-player input.
$u_{E,k}$	Evader or maximizing-player input.
A	Nominal discrete-time state matrix.
B_P, B_E	Pursuer and evader input matrices.
Q	Positive semidefinite state-penalty matrix.
R_P, R_E	Positive definite pursuer and evader input-weight matrices.
P	Quadratic value matrix.
L, K	Pursuer and evader state-feedback gain matrices.
b_P, b_E	Persistent realized-input offsets in the two command channels.
$\widehat{b}_P, \widehat{b}_E$	Estimated offsets used by the affine compensation layer.

2.3 Pursuer, Evader, and State-Space Definitions

The minimizing and maximizing players are interpreted as a pursuer and an evader. Under this interpretation, the pursuer attempts to reduce a selected relative-state error, while the evader attempts to preserve or enlarge that error. The interpretation follows from the standard zero-sum pursuit-evasion structure in which both players affect the same engagement state but act with opposing objectives [1, 2].

For the current reduced-order system, an auxiliary engagement output can be expressed by

$$y_k = Cx_k. \tag{2.2}$$

In addition, the verified MATLAB benchmark uses

$$C = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}. \tag{2.3}$$

Hence, $y_k = x_{1,k} - x_{3,k}$ can be interpreted as a reduced relative engagement variable. The output is useful for linking the state-space benchmark to pursuit-evasion language; however, it does not define the state-penalty matrix in this implementation. The numerical studies use $Q = I_3$, so all three states are penalized directly in the quadratic game objective.

This interpretation is intentionally more limited than a geometric vehicle capture condition. In a complete planar pursuit-evasion model, capture may be defined through the Euclidean separation between two position vectors. In the current linear benchmark, the output given in (2.2) is a local engagement proxy that supports interpretation and reporting. As a result, this chapter does not claim that $y_k = 0$ is identical to physical contact between two vehicles. Instead, it is a reduced-order quantity that allows the four algorithms and the command-offset extension to be studied and examined without changing the original benchmark matrices.

The pursuer and evader policies are both calculated from the same game state. Thus, the system uses a feedback information pattern rather than open-loop input sequences. At each time step, the current state is supplied to both policies, each player generates a command, and the two realized inputs jointly determine the next state. This feedback structure is central to the later policy iteration and Q-learning derivations since both methods seek stationary gain matrices rather than one-time input trajectories.

2.4 Quadratic Game Objective and Saddle-Point Policies

The infinite-horizon performance index is expressed by the equation

$$J(x_0, u_P, u_E) = \sum_{k=0}^{\infty} \left(x_k^\top Q x_k + u_{P,k}^\top R_P u_{P,k} - u_{E,k}^\top R_E u_{E,k} \right), \tag{2.4}$$

where $Q \succeq 0, R_P \succ 0$, and $R_E \succ 0$. The pursuer aims to minimize J, while the evader seeks to maximize the same quantity. As a result, the saddle objective can be written conceptually as

$$J^*(x_0) = \min_{u_P} \max_{u_E} J(x_0, u_P, u_E). \tag{2.5}$$

Notice that the sign structure in Equation (2.4) is essential. The pursuer is penalized for state deviation and its own control effort. The evader appears with a negative input penalty

because it acts as the maximizing player. Although no explicit actuator bounds are imposed in the nominal LQ game, the maximizing direction must satisfy the required concavity so that the saddle problem remains finite and well-posed. Rather, R_E and the saddle-point curvature conditions limit the maximizing direction and determine whether the stage game is well posed. In other words, the negative sign creates the adversarial objective, while the matrix conditions later prevent the maximization from becoming unbounded.

For the verified MATLAB benchmark, the weights are given by

$$Q = I_3, \quad R_P = 1, \quad R_E = \gamma^2, \quad \gamma = 1. \tag{2.6}$$

The parameter γ is consistent with the H_∞ interpretation of the source zero-sum game, but the present project report uses pursuer-evader language such that the same minimizing and maximizing channels that can be interpreted within a pursuit-evasion setting [5, 33].

The objective in (2.4) is evaluated using the inputs that actually enter the plant. This distinction becomes important after command bias is introduced. A controller may compute a nominal command, but the cost accumulated by the physical or simulated plant relies on the realized input in the state equation. Consequently, the later comparisons use the same cost definition for the nominal, biased, and compensated trajectories while changing only the relationship between commanded and realized inputs.

Quadratic Value Function and Feedback Saddle Policies

For a stabilizing linear-quadratic zero-sum game, the value function is assumed to have the quadratic form

$$V(x_k) = x_k^\top P x_k, \tag{2.7}$$

where $P = P^\top \succeq 0$ denotes the game value matrix. Then, the feedback policies are represented by

$$u_{P,k} = L x_k, \quad u_{E,k} = K x_k, \tag{2.8}$$

where $L \in \mathbb{R}^{m_P \times n}$ and $K \in \mathbb{R}^{m_E \times n}$. The sign convention in (2.8) is retained from the MATLAB source code. Therefore, any negative feedback direction is contained in the numerical values of L and K rather than written as an additional minus sign outside the gains.

Substituting Equation (2.8) into Equation (2.1) gives the nominal closed-loop system expressed by

$$x_{k+1} = A_{\text{cl}} x_k, \quad A_{\text{cl}} = A + B_P L + B_E K. \tag{2.9}$$

A stabilizing saddle solution requires A_{cl} to be Schur stable, meaning that every eigenvalue lies strictly inside the unit circle. This condition is then expressed as

$$\rho(A_{\text{cl}}) < 1, \tag{2.10}$$

where $\rho(\cdot)$ denotes the spectral radius.

The saddle policies must also satisfy the minimax ordering. For every admissible state, the pursuer policy should minimize the value against the evader policy, while the evader policy should

maximize the value against the pursuer policy. Conceptually, this is expressed as follows

$$J(x_0; L, K') \leq J(x_0; L, K) \leq J(x_0; L', K) \tag{2.11}$$

for admissible alternative gains L' and K'. The exact inequality depends on the policy class and initial state; however, the expression captures the central interpretation. More specifically, neither player can improve its objective through a unilateral admissible change when the other player holds the saddle policy fixed [1, 3].

The value matrix P is not only a cost summary. It also connects the model-based and model-free sections of this project report. In the model-based methods, P is evaluated or updated directly from the system matrices. In the model-free methods, the action-value matrix H is estimated from data, after which the policy gains and an equivalent state-value relationship are recovered. Thus, P and H represent two views of the same nominal game.

2.5 Equivalent Nominal Value Representations

The four algorithms considered in this project report aim to seek the same nominal saddle solution through two equivalent quadratic representations. The model-based algorithms operate directly on the state-value matrix P, while the model-free algorithms estimate a joint state-action matrix H. This section defines those common mathematical objects without reproducing the complete iteration procedures that will be developed in Chapter 3 and Chapter 4.

Generalized Algebraic Riccati Game Target

The Game Algebraic Riccati Equation (GARE) provides the steady-state model-based solution. To state the equation compactly, define the block game matrix as follows

$$\mathcal{G}(P) = \begin{bmatrix} R_P + B_P^\top P B_P & B_P^\top P B_E \\ B_E^\top P B_P & B_E^\top P B_E - R_E \end{bmatrix} \tag{2.12}$$

Then, the stacked gain matrix is

$$\begin{bmatrix} L \\ K \end{bmatrix} = -\mathcal{G}(P)^{-1} \begin{bmatrix} B_P^\top P A \\ B_E^\top P A \end{bmatrix}. \tag{2.13}$$

In the numerical implementation, 2.13 is evaluated through a linear solve rather than by explicitly forming the inverse. Next, the value matrix satisfies the following equation

$$P = Q + A^\top P A - \begin{bmatrix} A^\top P B_P & A^\top P B_E \end{bmatrix} \mathcal{G}(P)^{-1} \begin{bmatrix} B_P^\top P A \\ B_E^\top P A \end{bmatrix}. \tag{2.14}$$

Equation (2.14) is the common nominal target for the two model-based algorithms and the corresponding model-free Q-learning procedures [3].

The block matrix in Equation (2.12) must be numerically invertible. However, invertibility alone is not sufficient. Now, define

$$H_{PP} = R_P + B_P^\top P B_P, \tag{2.15}$$
$$H_{PE} = B_P^\top P B_E, \quad H_{EP} = B_E^\top P B_P, \tag{2.16}$$
$$H_{EE} = B_E^\top P B_E - R_E. \tag{2.17}$$

The minimizing-player block must be positive definite,

$$H_{PP} \succ 0, \tag{2.18}$$

and the maximizing-player Schur complement must be negative definite,

$$S_E = H_{EE} - H_{EP} H_{PP}^{-1} H_{PE} \prec 0. \tag{2.19}$$

The first condition ensures local convexity in the minimizing-player input. Then the second ensures local concavity in the maximizing-player input after the minimizing direction is accounted for. These conditions distinguish a valid saddle point from a stationary point that has the wrong curvature.

An equivalent closed-loop policy evaluation identity is

$$P = Q + L^\top R_P L - K^\top R_E K + A_{\mathrm{cl}}^\top P A_{\mathrm{cl}}. \tag{2.20}$$

This form is especially useful for policy iteration, since the gains remain fixed during the policy evaluation step. Additionally, it is also useful for verification, since a converged P, L, and K should satisfy both the GARE form as well as the closed-loop Lyapunov form within numerical tolerance. The detailed usage of (2.20) in policy iteration, and the repeated Riccati mapping used in value iteration, are presented in Sections 2.6 and 2.7.

2.6 Nominal GARE Policy Iteration

Admissible Initialization and Fixed-Policy Evaluation

Policy iteration commences with an admissible gain pair (L^0, K^0). At iteration i, the current closed-loop matrix is

$$A^i = A + B_P L^i + B_E K^i. \tag{2.21}$$

In order for the infinite-horizon policy evaluation to be meaningful, A_i must be Schur stable; that is,

$$\rho(A^i) < 1. \tag{2.22}$$

This is a required condition before the value that is associated with the current policy pair can be evaluated. If an unstable pair were inserted into the infinite-horizon Lyapunov equation, then the returned matrix would not represent a finite quadratic value for the closed-loop game.

For an admissible pair, define the fixed-policy stage matrix given by

$$S^i = Q + (L^i)^\top R_P L^i - (K^i)^\top R_E K^i. \tag{2.23}$$

Then, the policy evaluation step solves

$$(A^i)^\top P^i A^i - P^i + S^i = 0, \tag{2.24}$$

or equivalently,

$$P^i = S^i + (A^i)^\top P^i A^i. \tag{2.25}$$

The equation can be interpreted by repeated substitution. When A^i is Schur stable,

$$P^i = \sum_{j=0}^{\infty} ((A^i)^\top)^j S^i (A^i)^j. \tag{2.26}$$

Thus, $x_0^\top P^i x_0$ is the accumulated quadratic game value that is generated by holding the current pursuer and evader policies as fixed. This representation clarifies why policy evaluation and policy improvement are different operations. The Lyapunov equation does not question whether (L^i, K^i) is optimal; rather, it measures the infinite-horizon consequence of continuing to use that pair.

The existence of the Lyapunov solution alone is not enough to establish a valid zero-sum policy. Since the stage matrix contains the negative evader penalty, S^i need not be positive definite even when Q, R_P, and R_E have their required signs. Thus, the interpretation of P^i remains tied to the zero-sum saddle structure rather than to a single-player positive-cost argument as the later curvature tests are what determine whether the stationary input pair has the correct convex-concave ordering.

Coupled Saddle-Policy Improvement

After P^i has been evaluated, the current state-action quadratic form can be expressed as

$$\begin{aligned} Q^i(x, u_P, u_E) = {} & x^\top (Q + A^\top P^i A)x + 2x^\top A^\top P^i B_P u_P + 2x^\top A^\top P^i B_E u_E \\ & + u_P^\top H_{PP}(P^i) u_P + 2u_P^\top H_{PE}(P^i) u_E + u_E^\top H_{EE}(P^i) u_E. \end{aligned} \tag{2.27}$$

Then the stationary conditions with respect to the minimizing and maximizing inputs are

$$\frac{\partial Q^i}{\partial u_P} = 0, \quad \frac{\partial Q^i}{\partial u_E} = 0. \tag{2.28}$$

After substituting $u_P = L^{i+1}x$ and $u_E = K^{i+1}x$, it follows that these conditions produce

$$G(P^i) \begin{bmatrix} L^{i+1} \\ K^{i+1} \end{bmatrix} = -F(P^i). \tag{2.29}$$

Therefore, the gain pair is improved simultaneously. The pursuer update relies on the evader coupling blocks, and the evader update depends on the pursuer coupling blocks. This is the direct algebraic reason that the iteration cannot be interpreted as two independent regulator updates acting on the same plant.

Moreover, the source code algorithm expresses similar improvement through Schur complements. With all blocks evaluated at P^i,

$$L^{i+1} = -\left(H_{PP} - H_{PE}H_{EE}^{-1}H_{EP}\right)^{-1}\left(H_{Px} - H_{PE}H_{EE}^{-1}H_{Ex}\right), \tag{2.30}$$

$$K^{i+1} = -\left(H_{EE} - H_{EP}H_{PP}^{-1}H_{PE}\right)^{-1}\left(H_{Ex} - H_{EP}H_{PP}^{-1}H_{Px}\right). \tag{2.31}$$

These expressions make the elimination order more visible. The first equation accounts for the maximizing direction while forming the minimizing update, and the second accounts for the minimizing direction while forming the maximizing update. When the relevant blocks are nonsingular, it follows that the Schur complement formulas and the joint block solve produce the same gain pair.

The policy improvement state is only meaningful when the local game is well-posed. In particular, the pursuer block needs to satisfy

$$H_{PP}(P^i) \succ 0, \tag{2.32}$$

and the evader Schur complement must satisfy

$$S_E(P^i) = H_{EE}(P^i) - H_{EP}(P^i)H_{PP}(P^i)^{-1}H_{PE}(P^i) < 0. \tag{2.33}$$

The first inequality gives positive curvature in the minimizing input, while the second gives negative curvature in the maximizing direction after the pursuer input has been eliminated. A stationary solution that violates either sign condition is not the saddle point required by the game objective.

2.7 Nominal GARE Value Iteration

Riccati Optimality Mapping from the One-Step Game

Value iteration approaches the same GARE solution without solving a complete fixed-policy Lyapunov equation at every step. Suppose that the current value approximation is given by the expression

$$V^i(x) = x^\top P^i x. \tag{2.34}$$

The one-step dynamic programming expression is

$$\begin{aligned} V^{i+1}(x) = \min_{u_P}\max_{u_E}\Big[&x^\top Q x + u_P^\top R_P u_P - u_E^\top R_E u_E \\ &+ (Ax + B_P u_P + B_E u_E)^\top P^i (Ax + B_P u_P + B_E u_E)\Big]. \end{aligned} \tag{2.35}$$

Then, the stationary input pair that is associated with P^i satisfies

$$G(P^i)\begin{bmatrix} L^i \\ K^i \end{bmatrix} = -F(P^i). \tag{2.36}$$

Substitution of the stationary inputs into the one-step expression produces the Riccati mapping

$$\mathcal{T}(P^i) = Q + A^\top P^i A - F(P^i)^\top G(P^i)^{-1}F(P^i), \tag{2.37}$$

and the value update is

$$P^{i+1} = \mathcal{T}(P^i). \tag{2.38}$$

The above derivation explains why value iteration is not merely policy iteration written with different indices. Policy iteration evaluates the infinite-horizon value of a fixed gain pair, while value iteration performs one optimality backup with the current value approximation and advances directly to the next matrix.

An intermediate value iteration matrix does not necessarily equal the complete infinite-horizon value of the gain pair displayed at that same iteration. Moreover, its meaning is instead tied to repeated application of the optimality operator. This difference affects how intermediate stability is interpreted. Policy iteration requires an admissible current pair before each Lyapunov evaluation. On the other hand, value iteration requires the game blocks to remain well-posed during each Riccati step and requires the final recovered gain pair to satisfy the stabilizing saddle conditions.

Gain Recovery and Iterate Definition

After P^{i+1} has been formulated, the gain pair that is associated with the updated value matrix is recovered from

$$G(P^{i+1}) \begin{bmatrix} L^{i+1} \\ K^{i+1} \end{bmatrix} = -F(P^{i+1}). \tag{2.39}$$

The recovery equation has the same block structure that is used by policy improvement; moreover, what changes is the origin of the value matrix that is inserted into the blocks. In policy iteration, P^i is obtained by evaluating the current policies. In value iteration, P^{i+1} is obtained by applying the optimality map to the previous value approximation.

The gain sequence is still important even though value iteration is organized around the value matrix. Monitoring the recovered gains will show whether the policy quantities approach stable values as the Riccati sequence converges. In addition, it also permits the same final checks used by policy iteration; that is, the closed-loop spectral radius, the pursuer curvature, the evader Schur complement, and the block-matrix conditioning. These checks do not turn value iteration into policy iteration, as they determine whether the stationary policies implied by the current value approximation are numerically and game-theoretically meaningful.

The final convergence test again requires more than a small iteration change. More specifically, the converged matrix must satisfy the expression,

$$P = \mathcal{T}(P), \tag{2.40}$$

and the recovered gains must generate a Schur stable closed-loop with the correct saddle curvature. When policy iteration and value iteration both converge to the same stabilizing GARE solution, then their final matrices and gains should agree within numerical tolerance. In addition, their intermediate histories may differ since one sequence is built from fixed-policy evaluations and the other from repeated optimality backups.

Chapter 3

Model-Based Bias Compensation: Policy and Value Iteration

3.1 Persistent Command-Channel Offset Model

The nominal algorithms assume that the command generated by each policy is the input that enters the plant. The disturbance-aware extension removes this idealized equivalence by separating the commanded input from the realized input. This a crucial distinction since a policy can be mathematically correct for the nominal game while the executed plant still experiences a persistent offset. Moreover, the offset does not require the GARE or the quadratic Q-function to be replaced. Instead, it introduces an affine execution-side term that must be identified and compensated after the nominal saddle policies have been obtained.

For the pursuer, the realized input can be expressed as

$$u_{P,k}^{\mathrm{act}} = u_{P,k}^{\mathrm{cmd}} + b_P, \tag{3.1}$$

and for the evader,

$$u_{E,k}^{\mathrm{act}} = u_{E,k}^{\mathrm{cmd}} + b_E, \tag{3.2}$$

where $b_P \in \mathbb{R}^{m_P}$ and $b_E \in \mathbb{R}^{m_E}$ are constant or slowly varying command-channel offsets over the calibration and simulation windows. The offsets are matched disturbances since each term enters through the same input matrix as the corresponding player command. Therefore, the realized plant is given by

$$x_{k+1} = Ax_k + B_P u_{P,k}^{\mathrm{act}} + B_E u_{E,k}^{\mathrm{act}}. \tag{3.3}$$

If the nominal feedback commands are applied without compensation,

$$u_{P,k}^{\mathrm{cmd}} = Lx_k, \quad u_{E,k}^{\mathrm{cmd}} = Kx_k, \tag{3.4}$$

then the closed-loop dynamics become

$$x_{k+1} = A_{\mathrm{cl}} x_k + B_P b_P + B_E b_E. \tag{3.5}$$

The constant affine forcing term is

$$d_b = B_P b_P + B_E b_E. \tag{3.6}$$

And when $I - A_{\mathrm{cl}}$ is nonsingular, the corresponding biased equilibrium is

$$x_\infty^{\mathrm{bias}} = (I - A_{\mathrm{cl}})^{-1} d_b. \tag{3.7}$$

Equation (3.7) distinguishes stability from regulation. A Schur-stable feedback matrix can still drive the realized plant toward a nonzero equilibrium when the command offsets remain uncompensated. As a result, the disturbance-aware problem is not created by a failure of the nominal Riccati or Q-learning solution. It is created by the difference between the policy command and the input that is actually applied. This observation supplies the mathematical reasoning for adding an estimation and compensation layer while retaining the nominal gain pair.

The same offset model can represent several execution-side effects. A repeatable acceleration error, steering-center offset, motor-command conversion error, actuator calibration mismatch, or persistent input-reconstruction error can be approximated by b_P and b_E over a finite experiment. This project report does not claim that every fault is constant. Rather, the constant-offset assumption provides a transparent first disturbance model whose influence can be derived exactly and whose compensation can be verified without changing the four source algorithms [44, 45].

3.2 Model-Based Transition Residual Estimation

For the model-based case, define the one-step transition residual as follows

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}. \tag{3.8}$$

Substituting the realized-input model gives

$$r_k = \underbrace{\begin{bmatrix} B_P & B_E \end{bmatrix}}_{G_b} \underbrace{\begin{bmatrix} b_P \\ b_E \end{bmatrix}}_{b} + v_k, \tag{3.9}$$

where v_k collects transition-measurement noise and unmodeled effects. For a constant bias over N_{cal} samples, the sample-mean residual satisfies

$$\bar{r} = G_b b + \bar{v}. \tag{3.10}$$

The least-squares estimate is expressed by

$$\widehat{b} = G_b^{\dagger} \bar{r}, \tag{3.11}$$

where $(\cdot)^{\dagger}$ denotes the Moore-Penrose pseudoinverse. In the MATLAB implementation, the estimate is obtained through a linear least-squares solve rather than by explicitly forming an inverse. This formulation uses the known plant matrices only in the model-based calibration stage and leaves the original GARE policy or value update unchanged.

The two individual offsets are identifiable from the transition residual only when

$$\mathrm{rank}(G_b) = m_P + m_E. \tag{3.12}$$

If the two input directions are linearly dependent, the net affine disturbance d_b may still be identifiable even when the separate player offsets are not. Thus, this rank condition is a physical identifiability requirement rather than only a numerical convenience.

For independent isotropic transition noise with covariance $\sigma_v^2 I$, the covariance of the sample-mean least-squares estimate is

$$\mathrm{Cov}(\widehat{b}) = \frac{\sigma_v^2}{N_{\mathrm{cal}}}(G_b^\top G_b)^{-1}, \tag{3.13}$$

provided that G_b has full column rank.

3.3 Affine Compensation and Residual Closed-Loop Behavior

After the offsets are estimated, then the policy-generated inputs are modified before they enter the biased command channels. The compensated commands are as follows

$$u_{P,k}^{\mathrm{cmd}} = Lx_k - \widehat{b}_P, \tag{3.14}$$
$$u_{E,k}^{\mathrm{cmd}} = Kx_k - \widehat{b}_E. \tag{3.15}$$

Thus, it follows that the realized inputs are

$$u_{P,k}^{\mathrm{act}} = Lx_k - \widehat{b}_P + b_P, \tag{3.16}$$
$$u_{E,k}^{\mathrm{act}} = Kx_k - \widehat{b}_E + b_E. \tag{3.17}$$

Next, define the residual bias errors

$$\widetilde{b}_P = b_P - \widehat{b}_P, \quad \widetilde{b}_E = b_E - \widehat{b}_E. \tag{3.18}$$

Hence, substitution into the plant gives

$$x_{k+1} = A_{\mathrm{cl}}x_k + B_P\widetilde{b}_P + B_E\widetilde{b}_E. \tag{3.19}$$

Equation (3.19) contains the complete disturbance-aware interpretation. When the estimates are exact, it follows that

$$\widehat{b}_P = b_P, \quad \widehat{b}_E = b_E, \tag{3.20}$$

the residual offsets vanish and

$$x_{k+1} = A_{\mathrm{cl}}x_k. \tag{3.21}$$

As a result, exact compensation recovers the nominal closed-loop dynamics without recomputing P, H, L, or K. The exact case is not presented as an implementable solution when the offsets are unknown. It is a verification. So, if exact cancellation does not reproduce the nominal trajectory to numerical precision, the command and realized-input bookkeeping is inconsistent.

When the estimates contain error, the compensated plant remains subject to a smaller affine term. Provided that $I - A_{\mathrm{cl}}$ is nonsingular, its residual equilibrium is

$$x_{\infty}^{\mathrm{comp}} = (I - A_{\mathrm{cl}})^{-1} \left(B_P \widetilde{b}_P + B_E \widetilde{b}_E \right). \tag{3.22}$$

This expression separates the three quantities that determine the final regulation error. The first is the estimation error itself, and the second is the direction through which each residual enters the state. The third is the closed-loop amplification that is contained in $(I - A_{\mathrm{cl}})^{-1}$. Thus, a small scalar bias-estimation error does not automatically guarantee an equally small state error. A nearly marginal closed-loop is able to amplify a constant residual more strongly than a well-damped closed-loop.

The deviation from the nominal trajectory is defined as follows

$$d_k = x_k^{\mathrm{case}} - x_k^{\mathrm{nom}}. \tag{3.23}$$

For a matched initial condition, $d_0 = 0$, and the deviation recurrence is

$$d_{k+1} = A_{\mathrm{cl}} d_k + B_P \widetilde{b}_P + B_E \widetilde{b}_E. \tag{3.24}$$

Its finite-step solution is expressed as

$$d_k = \sum_{j=0}^{k-1} A_{\mathrm{cl}}^j \left(B_P \widetilde{b}_P + B_E \widetilde{b}_E \right). \tag{3.25}$$

Moreover, the MATLAB implementations compare this analytical recurrence with the simulated trajectory difference as the agreement to floating-point precision verifies that the observed deviation is caused by the intended affine forcing rather than by inconsistent simulation logic.

The affine compensation layer preserves the nominal feedback gains. This design decision is key to the claim of this project report. Each source algorithm computes L and K according to its original policy iteration or value iteration structure. Calibration estimates b_P and b_E, and execution subtracts those estimates before the commands enter the biased plant. Therefore, the extension is traceable to the original algorithms while remaining mathematically sufficient to remove a matched constant offset.

3.4 Disturbance-Aware Extension of GARE Policy Iteration

Reasoning for Application of the Extension after Nominal Convergence

The nominal policy iteration procedure assumes that the command produced by each feedback law is the input that enters the plant. The disturbance-aware model separates those quantities as follows:

$$u_{P,k}^{\mathrm{act}} = u_{P,k}^{\mathrm{cmd}} + b_P, \quad u_{E,k}^{\mathrm{act}} = u_{E,k}^{\mathrm{cmd}} + b_E. \tag{3.26}$$

When the uncompensated commands are $u_{P,k}^{\mathrm{cmd}} = Lx_k$ and $u_{E,k}^{\mathrm{cmd}} = Kx_k$, it follows that the realized dynamics become

$$x_{k+1} = A_{\mathrm{cl}} x_k + B_P b_P + B_E b_E. \tag{3.27}$$

The constant forcing term changes the equilibrium of the executed plant, but it does not change the linear part A_{cl}. Therefore, a nominally Schur-stable controller can remain stable while producing a persistent state displacement.

The offsets are not inserted into the Lyapunov policy evaluation equation nor into the saddle-gain improvement as doing so would define an affine optimal control problem with a different value function structure. This could be a potential research direction, but it is not explored here. This project report preserves the nominal zero-sum game and treats the offsets as execution-side mismatches that are estimated and canceled after the nominal gain pair has been obtained.

Moreover, this ordering also creates a clear diagnostic hierarchy. A failed GARE residual, curvature test, or spectral radius check indicates a problem within the algorithm. A valid nominal controller followed by a biased trajectory indicates a command-channel mismatch. The failure of exact compensation to reproduce the nominal trajectory indicates inconsistent command and realized-input bookkeeping. Keeping the layers separate allows the disturbance-aware study to attribute an observed error to the controller, the estimator, or the execution model rather than combining those causes into one modified Riccati equation.

Known-Model Transition Residual Estimation

During calibration, define the one-step transition residual expression given by

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}} \tag{3.28}$$

Substitution of the realized-input equations gives

$$r_k = G_b b + v_k, \quad G_b = \begin{bmatrix} B_P & B_E \end{bmatrix}, \quad b = \begin{bmatrix} b_P \\ b_E \end{bmatrix}, \tag{3.29}$$

where v_k collects transition-measurement noise and effects not represented by the constant-offset model. The residual has a simple interpretation; that is, the residual is the portion of the observed transition that cannot be explained by the nominal plant and the recorded commands.

For N_{cal} samples with constant offsets, the sample mean satisfies the expression

$$\bar{r} = G_b b + \bar{v}, \quad \bar{r} = \frac{1}{N_{\mathrm{cal}}} \sum_{k=1}^{N_{\mathrm{cal}}} r_k. \tag{3.30}$$

Then, the least-squares estimate is expressed by

$$\widehat{b} = \arg\min_{\beta} \|G_b \beta - \bar{r}\|_2^2 = G_b^{\dagger} \bar{r}. \tag{3.31}$$

A stacked formulation that uses all residuals produces the same estimate when the bias map is constant and the samples are equally weighted. The sampled-mean form makes the role of a persistent offset especially clear; moreover, repeated measurements reduce the random component of the residual while the constant component remains.

The individual offsets are only identifiable if they satisfy the expression

$$\mathrm{rank}(G_b) = m_P + m_E. \tag{3.32}$$

This is both a physical and numerical condition. More specifically, if B_P and B_E act as indis-

tinguishable directions, then the transition data may reveal the net forcing $B_P b_P + B_E b_E$ without revealing how that forcing should be assigned to the command channels. In that situation, exact cancellation of the net disturbance may still be possible under additional design choices; however, the separate pursuer and evader offsets are not uniquely determined by the residual equation alone.

Conditioning determines how strongly noise is amplified by the identification map. When G_b is full column rank, but nearly rank deficient, it follows that small transition errors can produce comparatively large changes in $\hat{b}$. For independent isotropic residual noise with a covariance $\sigma_v^2 I$, the sample-mean estimator covariance is given by

$$\mathrm{Cov}(\widehat{b}) = \frac{\sigma_\nu^2}{N_{\mathrm{cal}}} \left(G_b^\top G_b \right)^{-1}. \tag{3.33}$$

The expression separates three influences on estimator speed; more specifically, the transition-noise level, the calibration length and the geometry of the two input directions. If N_{cal} is increased, then it reduces the covariance by averaging, while poor directional separation in G_b increases the covariance through the inverse Gram matrix.

Affine Compensation and Residual Bias Dynamics

Now, partition $\hat{b} = \begin{bmatrix} \hat{b}_P & \hat{b}_E \end{bmatrix}^\top$. Then, the compensated commands are given by

$$u_{P,k}^{\mathrm{cmd}} = L x_k - \hat{b}_P, \quad u_{E,k}^{\mathrm{cmd}} = K x_k - \hat{b}_E. \tag{3.34}$$

Then, the realized inputs are given as

$$u_{P,k}^{\mathrm{act}} = L x_k + \tilde{b}_P, \quad u_{E,k}^{\mathrm{act}} = K x_k + \widetilde{b}_E, \tag{3.35}$$

where

$$\tilde{b}_P = b_P - \hat{b}_P, \quad \tilde{b}_E = b_E - \hat{b}_E. \tag{3.36}$$

Hence,

$$x_{k+1} = A_{\mathrm{cl}} x_k + B_P \tilde{b}_P + B_E \tilde{b}_E. \tag{3.37}$$

The compensation law does not remove the effect of estimation error as it replaces the original offsets with the residual offsets. This distinction is crucial since it links the estimator directly to the remaining state displacement.

Let x_k^{nom} denote the nominal trajectory and let x_k^{comp} denote the estimation-compensation trajectory from the same initial condition. The deviation given by

$$d_k = x_k^{\mathrm{comp}} - x_k^{\mathrm{nom}}, \tag{3.38}$$

which satisfies

$$d_{k+1} = A_{\mathrm{cl}} d_k + B_P \tilde{b}_P + B_E \tilde{b}_E, \quad d_0 = 0. \tag{3.39}$$

Hence,

$$d_k = \sum_{j=0}^{k-1} A_{\mathrm{cl}}^{j} \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{3.40}$$

When A_{cl} is Schur stable, then

$$d_\infty = (I - A_{\mathrm{cl}})^{-1} \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{3.41}$$

(3.41) shows why the same scalar estimation error can have different state consequences under different closed-loop dynamics. The residual bias is first mapped through the input direction and is then amplified or attenuated by $(I - A_{\mathrm{cl}})^{-1}$.

It then follows immediately that a useful norm bound can be given by

$$\|d_\infty\|_2 \leq \|(I - A_{\mathrm{cl}})^{-1}\|_2 \left(\|B_P\|_2 \|\tilde{b}_P\|_2 + \|B_E\|_2 \|\tilde{b}_E\|_2 \right). \tag{3.42}$$

The bound is not used as a tight performance prediction; rather, its role is interpretive and serves to separate estimate accuracy, input-direction magnitude, and closed-loop sensitivity, thereby preventing compensation quality from being judged solely by bias error.

Exact compensation sets $\hat{b}_P = b_P$ and $\hat{b}_E = b_E$. In that case, $\tilde{b}_P = \tilde{b}_E = 0$ and the realized dynamics can be reduced to

$$x_{k+1} = A_{\mathrm{cl}} x_k. \tag{3.43}$$

Thus, the exact case is a verification condition, as it does not assume that unknown offsets can be known in practice. In addition, the exact case questions whether the command transformation and realized-input equations are internally consistent. If exact cancellation fails to reproduce the nominal trajectory, then the discrepancy cannot be attributed to estimator noise.

Algorithm 1 Disturbance-Aware Extension of GARE Policy Iteration

Phase I: Compute and verify the nominal policy-iteration solution.
Execute the nominal GARE policy-iteration procedure until the value matrix and gain pair have converged.
Verify the GARE residual, Lyapunov residual, closed-loop spectral radius, saddle curvature, and game-matrix conditioning.
Phase II: Estimate the persistent command offsets.
Record x_k, x_{k+1}, $u_{P,k}^{\mathrm{cmd}}$, and $u_{E,k}^{\mathrm{cmd}}$ during calibration.
Form $r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}$.
Compute $\widehat{b} = G_b^{\dagger}\bar{r}$ through a least-squares solve.
Verify the rank and conditioning of G_b and determine the estimator covariance under the stated noise model.
Phase III: Apply the four matched execution cases.
Evaluate nominal operation with zero offsets.
Evaluate uncompensated operation with the true offsets added to the realized channels.
Evaluate estimated compensation using $Lx_k - \widehat{b}_P$ and $Kx_k - \widehat{b}_E$.
Evaluate exact compensation using $Lx_k - b_P$ and $Kx_k - b_E$.
Phase IV: Verify the disturbance-aware layer.
Report the offset-estimation errors and the residual affine forcing $B_P\widetilde{b}_P + B_E\widetilde{b}_E$.
Compare the observed state deviation with the analytical affine recurrence.
Verify that exact compensation reproduces the nominal trajectory to numerical precision.

3.5 Disturbance-Aware Extension of GARE Value Iteration

The command offsets are properties of the execution channels, not properties of the numerical route that is used to calculate the nominal gain pair. As a result, the value iteration extension uses the same residual estimator and affine command law developed for policy iteration as only the nominal computation phase changes. After value iteration converges, then the final matrix P and recovered gains (L, K) are verified through the GARE residual, spectral radius, saddle curvature, and game matrix conditioning. Then, calibration forms

$$r_k = x_{k+1} - Ax_k - B_P u_{P,k}^{\mathrm{cmd}} - B_E u_{E,k}^{\mathrm{cmd}}, \tag{3.44}$$

solves for $\hat{b}$, and applies

$$u_{P,k}^{\mathrm{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\mathrm{cmd}} = Kx_k - \hat{b}_E. \tag{3.45}$$

The compensated closed-loop is again given by

$$x_{k+1} = A_{\mathrm{cl}}x_k + B_P\tilde{b}_P + B_E\tilde{b}_E. \tag{3.46}$$

Hence, the theoretical relationship between estimation error and residual state displacement is identical for the two model-based methods once their final gains are fixed.

Using the same estimator and compensation law is essential for a fair comparison. If policy iteration and value iteration were paired with different calibration procedures, a later difference in recovery could not be attributed cleanly to the nominal iteration structure. Holding the disturbance-aware layer fixed ensures that the model-based comparison asks a single question. That

Algorithm 2 Disturbance-Aware Extension of GARE Value Iteration

The nominal core is the GARE value-iteration procedure described above. The remaining phases match the policy-iteration extension.
Phase I: Compute and verify the nominal value-iteration solution.
Execute the nominal GARE value-iteration procedure until the Riccati sequence and recovered gains have converged.
Verify the GARE residual, closed-loop spectral radius, saddle curvature, and game-matrix conditioning.
Phase II: Estimate the persistent command offsets.
Collect calibration transitions and the corresponding commanded inputs.
Form the known-model residuals and compute $\hat{b} = G_b^{\dagger}\bar{r}$.
Verify the rank and conditioning of G_b and determine the estimator covariance under the stated noise model.
Phase III: Apply the matched execution cases.
Evaluate nominal and uncompensated operation with the converged value-iteration gains.
Evaluate estimated compensation using $Lx_k - \hat{b}_P$ and $Kx_k - \hat{b}_E$.
Evaluate exact compensation using the true offsets.
Phase IV: Verify the disturbance-aware layer.
Report the offset-estimation errors, residual affine forcing, and predicted residual equilibrium.
Compare the simulated deviation with the analytical recurrence.
Verify that exact compensation reproduces the nominal trajectory to numerical precision.

is, how do two established Riccati procedures reach and verify the nominal saddle solution before being subjected to the same execution mismatch?

3.6 Model-Based GARE Policy Iteration Results

Nominal Convergence and Saddle Validity

The GARE policy iteration code converged in four outer iterations. Each outer step first evaluated the complete infinite-horizon value of the current admissible pair through a discrete Lyapunov equation and then updated the pursuer and evader gains through the coupled saddle solve. Figure 3.1 demonstrates that the value matrix and gain changes decreased by several orders of magnitude over these four steps. The rapid outer convergence is consistent with policy iteration; however, it should not be interpreted as requiring only four scalar calculations as each step includes a complete fixed policy matrix evaluation.

The residuals in Table 3.1 are close to floating-point precision. The spectral radius is below unity, the pursuer curvature is positive, and the reduced evader curvature is negative. These checks demonstrate that the four-step sequence did not merely stop changing; rather, it reached the stabilizing saddle solution of the stated game.

Known-Model Offset Estimation

The model-based calibration estimated the pursuer offset as $\widehat{b}_P = 0.0400013670$, and the evader offset as $\widehat{b}_E = -0.0102874376$. The pursuer percentage error was approximately 0.00342%, while the evader percentage error was approximately 2.874%. The difference in accuracy is consistent with

Table 3.1: Nominal Numerical Validation for Model-Based GARE Policy Iteration

Diagnostic	**Observed Value**
Converged	Yes
Outer Iterations	4
Closed-Loop Spectral Radius	0.9819707273
GARE Residual	6.4048×10^{-15}
Normalized GARE Residual	2.2076×10^{-16}
Lyapunov Residual	4.3512×10^{-15}
Normalized Lyapunov Residual	1.4998×10^{-16}
Game-Matrix Reciprocal Condition Number	0.56579379
Minimum Pursuer Curvature	1.761840343
Maximum Evader Curvature	−0.998509990

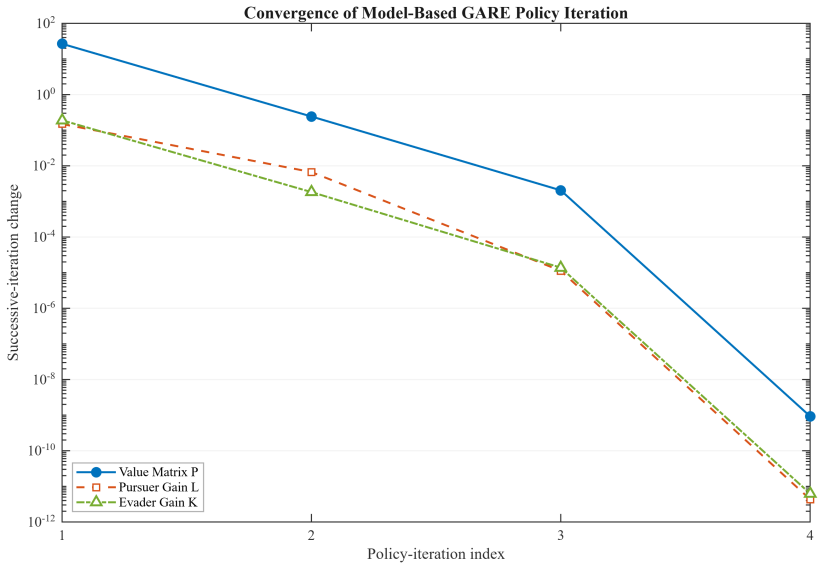


Figure 3.1: Model-Based GARE Policy Iteration Convergence

the geometry of the matrix $\begin{bmatrix} B_P & B_E \end{bmatrix}$. The calibration residual must separate two offsets through plant input directions whose condition number is approximately 91.05. Hence, the evader channel is recovered with a broader sampling distribution even though the same transition noise-level acts on the state equation.

Figure 3.2 shows the Monte Carlo distributions for the known model estimator. The pursuer estimates are tightly concentrated, whereas the evader estimates have a wider spread since the two offsets are inferred through the plant input directions. The primary estimate lies within the expected distribution in both channels, confirming the predicted direction-dependent uncertainty.

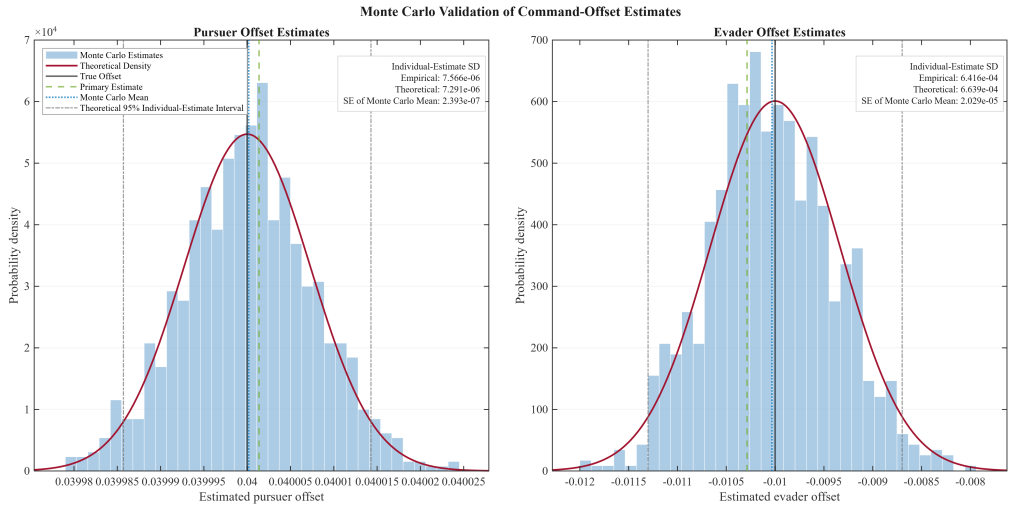


Figure 3.2: Known-Model Offset Estimator for GARE Policy Iteration

Trajectory Recovery and Analytical Verification

Without compensation, the persistent offsets produced a maximum Q-weighted state deviation of 4.12863×10^{-2}. Estimated compensation reduced the maximum deviation to 1.13499×10^{-4}, which corresponds to a reduction of approximately 99.7251%. Exact compensation reproduced the nominal trajectory to approximately 1.39×10^{-17}. The analytical affine recurrence agreed with the independently simulated deviation to approximately 2.69×10^{-15} in the uncompensated case as 4.68×10^{-15} in the estimated compensation case.

Figure 3.3 shows the physical interpretation of these values. The uncompensated deviation approaches a nonzero steady displacement since the stable homogeneous dynamics are driven by a constant affine term. Estimated compensation leaves a much smaller affine term, so the response approaches a correspondingly smaller steady displacement. The result does not require a change to L or K; rather, it follows from translating the command by the estimated offsets.

The signed finite-window nominal cost was $J_{\text{nom}} = 3169.85881235989$. Without compensation, the cost increased to $J_{\text{uncomp}} = 3171.20561554776$, which is a difference of approximately 1.3468. Estimated compensation produced $J_{\text{est}} = 3169.85876338411$, which differs from the nominal by approximately -4.90×10^{-5}. Exact compensation reproduced the nominal cost to the reported precision. The small negative difference in the estimated case does not imply that imperfect compensation outperformed the saddle solution. Rather, it is a finite-window signed cost effect caused by the small residual offsets acting on both minimizing and maximizing input terms. The trajectory deviation and exact compensation checks provide the more direct recovery interpretation.

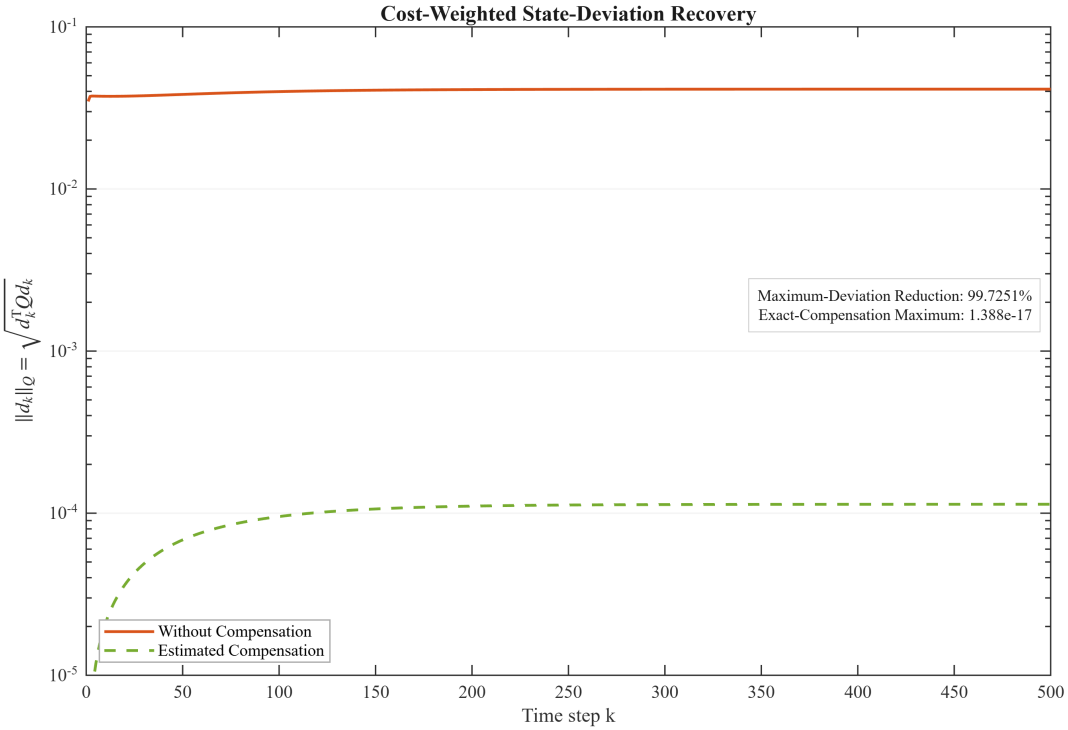


Figure 3.3: GARE Policy Iteration State Deviation Recovery

Command-Channel Behavior

Figure 3.4 separates the policy output, the compensated command, and the realized input during late-time portion of the simulation. In the pursuer channel, the command is shifted downward by the estimated positive offset before the true positive offset is added by the command channel. In the evader channel, the command is shifted in the opposite direction since the true offset is negative. In both cases, the realized estimated compensation input nearly overlaps the nominal input, while the uncompensated input retains a persistent displacement.

The small remaining separation is exactly the offset estimation residual. Taken together, Figures 3.1-3.4 establish the complete model-based policy iteration argument. More specifically, the nominal algorithm reaches a valid GARE saddle solution, the residual estimator recovers the offsets with the uncertainty predicted by the input geometry, and the affine correction restores the nominal execution behavior to the level permitted by the estimator error.

3.7 Model-Based GARE Value Iteration Results

Nominal Convergence and Saddle Validity

GARE value iteration approached the same nominal saddle solution through repeated applications of the Riccati optimality mapping; moreover, it converged in 633 value iteration steps. The longer sequence is not inconsistent with the four policy iteration updates since policy iteration solves a complete fixed-policy evaluation problem before each improvement, whereas value iteration advances the value approximation through one optimality mapping per step.

The residual magnitudes are consistent with the 10^{-10} stopping tolerance. The final spectral

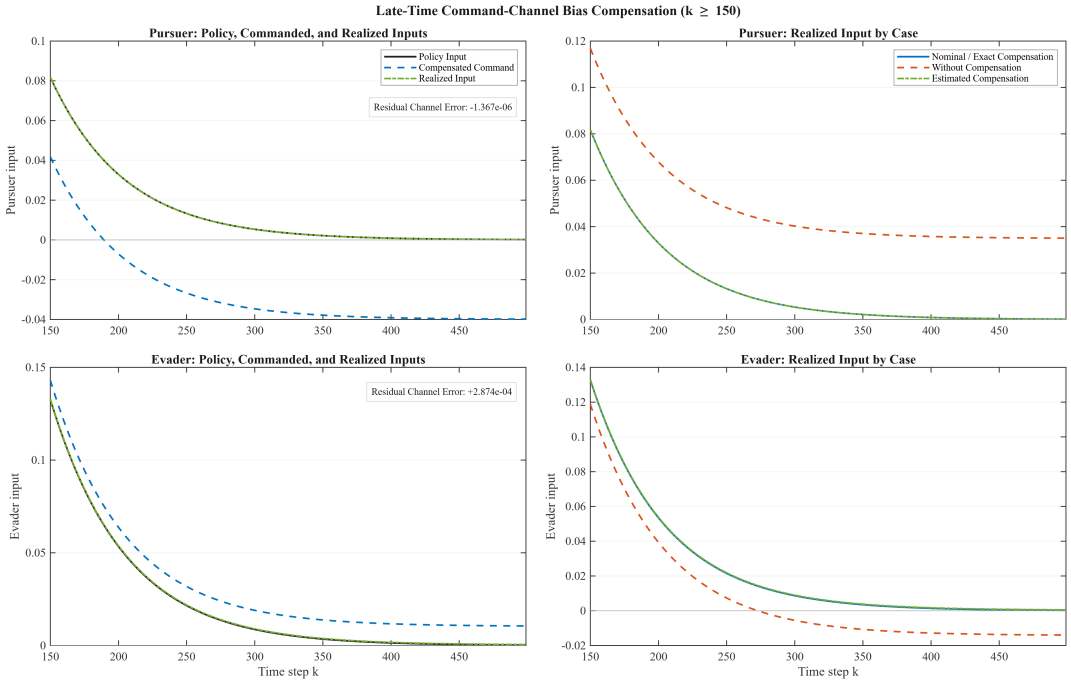


Figure 3.4: GARE Policy Iteration Command Channel Compensation

Table 3.2: Nominal Numerical Validation for Model-Based GARE Value Iteration

Diagnostic	**Observed value**
Converged	Yes
Value-Iteration Steps	633
Closed-Loop Spectral Radius	0.9819707273
GARE Residual	9.4437×10^{-11}
Normalized GARE Residual	3.2551×10^{-12}
Lyapunov Residual	9.4440×10^{-11}
Normalized Lyapunov Residual	3.2552×10^{-12}
Game-Matrix Reciprocal Condition Number	0.56579379
Minimum Pursuer Curvature	1.761840343
Maximum Evader Curvature	-0.998509990

radius, conditioning, and curvature values agree with the policy iteration result to the displayed precision, confirming that the two model-based algorithms reach the same controller by different numerical paths.

Figure 3.5 demonstrates an approximately geometric decrease in the value matrix and gain changes over the 633 updates. The nearly straight trends on the logarithmic scale indicate regular contraction over most of the iteration. The gain changes remain below the value matrix change near convergence, and all three sequences approach their selected numerical floors without reversal or sustained oscillation.

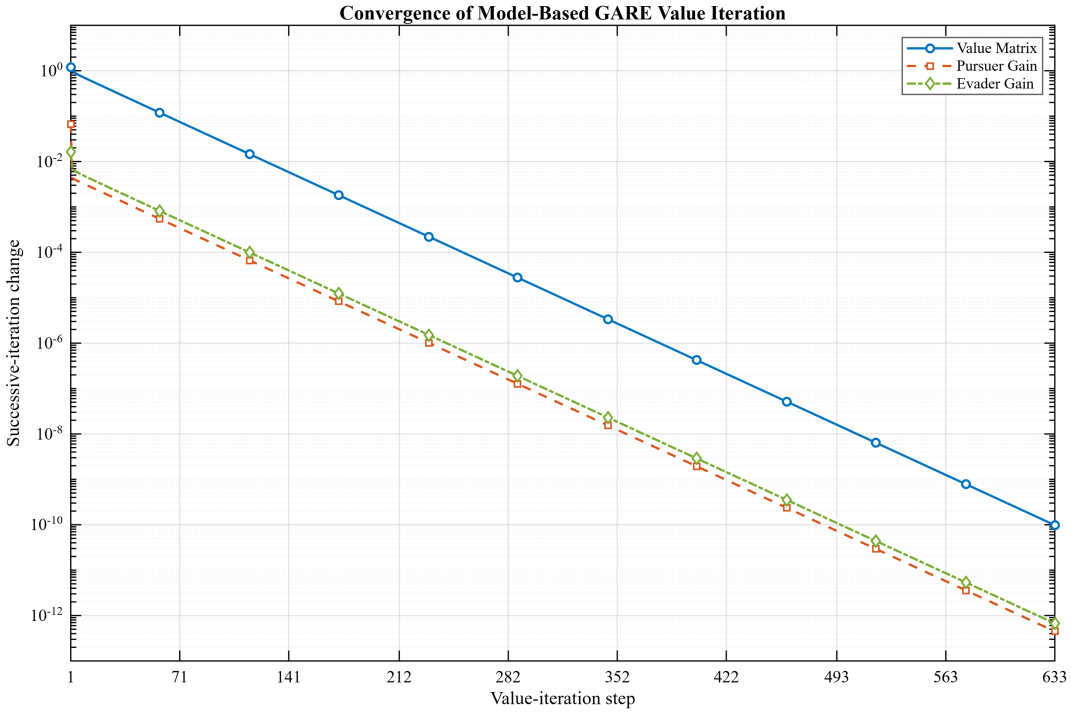


Figure 3.5: Model-Based GARE Value Iteration Convergence

Chapter 4

Model-Free Q-Learning Formulation, Algorithms, and Results

4.1 Model-Free Information Structure and Common Action-Value Objects

Measured Transition Tuples and Realized Stage Cost

The model-free learner operates on transition tuples of the form

$$\mathcal{D} = \left\{ \left(x_k, u_{P,k}^{\mathrm{act}}, u_{E,k}^{\mathrm{act}}, \ell_k, x_{k+1} \right) \right\}_{k=1}^{N}, \tag{4.1}$$

where $x_k \in \mathbb{R}^n$ denotes the measured game state, while $u_{P,k}^{\mathrm{act}} \in \mathbb{R}^{m_P}$ and $u_{E,k}^{\mathrm{act}} \in \mathbb{R}^{m_E}$ are the inputs that actually entered the plant, and x_{k+1} is the measured successor state. The corresponding one-step game cost is

$$\ell_k = x_k^\top Q x_k + (u_{P,k}^{\mathrm{act}})^\top R_P u_{P,k}^{\mathrm{act}} - (u_{E,k}^{\mathrm{act}})^\top R_E u_{E,k}^{\mathrm{act}}. \tag{4.2}$$

The pursuer minimizes the accumulated cost, while the evader maximizes it. The negative evader penalty preserves the minimax structure of the game and requires the same convex-concave curvature conditions that were used in the previous chapter.

The use of realized inputs in (4.1) and (4.2) is not a stylistic choice. A sampled Bellman equation associates a state-action pair with the transition and cost produced by that pair. If a command $u_{P,k}^{\mathrm{cmd}}$ is stored while the plant actually receives $u_{P,k}^{\mathrm{cmd}} + b_P$, then the recorded action does not match the observed successor state. The regression may still return a finite coefficient vector, but it is fitting internally inconsistent data. The learned quadratic form would then mix policy approximation error with an unmodeled labeling error at the command interface.

This observation gives the disturbance-aware extension a data-level role that does not arise in the previous chapter. For the model-based algorithms, the known matrices permit the offset to be reconstructed from a state-transition residual after nominal controller design. For Q-learning, the action recorded in the dataset is part of the Bellman feature vector. As a result, command realization must be handled before the data can be interpreted as samples of a state-action value function. Calibration and accurate action labeling do not alter the Bellman equation; rather, they ensure that the equation is applied to the transition that actually occurred.

Quadratic Joint State-Action Representation

Now, define the joint state-action vector that is given by

$$z_k = \begin{bmatrix} x_k \\ u_{P,k} \\ u_{E,k} \end{bmatrix} \in \mathbb{R}^{n_z}, \quad n_z = n + m_P + m_E. \tag{4.3}$$

For a linear-quadratic zero-sum game, the action-value function associated with a stationary policy pair is expressed as

$$Q(z_k) = z_k^\top H z_k, \quad H = H^\top. \tag{4.4}$$

Then, partition H according to the state, pursuer input, and evader input:

$$H = \begin{bmatrix} H_{xx} & H_{xP} & H_{xE} \\ H_{Px} & H_{PP} & H_{PE} \\ H_{Ex} & H_{EP} & H_{EE} \end{bmatrix}. \tag{4.5}$$

Symmetry implies that $H_{Px} = H_{xP}^\top, H_{Ex} = H_{xE}^\top$, and $H_{EP} = H_{PE}^\top$ in exact arithmetic. Moreover, these relationships are enforced when the coefficient vector is reconstructed into a matrix, since a small asymmetric numerical component has no independent meaning in quadratic form.

The representation in (4.4) is structurally matched to the game rather than selected as a generic nonlinear approximator. If a value matrix P were known, then the exact nominal action-value matrix would be

$$H(P) = \begin{bmatrix} Q + A^\top P A & A^\top P B_P & A^\top P B_E \\ B_P^\top P A & R_P + B_P^\top P B_P & B_P^\top P B_E \\ B_E^\top P A & B_E^\top P B_P & B_E^\top P B_E - R_E \end{bmatrix}. \tag{4.6}$$

The model-free algorithms do not reconstruct (4.6) by substituting A, B_P, B_E, and P. Rather, they estimate the independent entries of H from sampled Bellman relationships. Equation (4.6) is important as it establishes the target structure and explains why the learned input blocks have the same saddle interpretation as the GARE blocks in the previous chapter [4].

Since H is symmetric, it contains

$$n_H = \frac{n_z(n_z+1)}{2} \tag{4.7}$$

independent coefficients. Then, define a symmetric quadratic feature map $\phi : \mathbb{R}^{n_z} \to \mathbb{R}^{n_H}$ that contains each squared component of z and twice each distinct cross product. With a consistent ordering of the independent coefficients,

$$z^\top H z = \phi(z)^\top \theta, \tag{4.8}$$

where $\theta \in \mathbb{R}^{n_H}$ denotes the vector of independent entries of H. The Q-function is quadratic in the state and inputs, but linear in the unknown parameters. This distinction is what allows for exact least-squares recovery in the ideal linear-quadratic setting when the data provide sufficient excitation.

The doubled cross terms in $\phi(z)$ are bookkeeping rather than an additional model assumption.

For example, if $H_{ij} = H_{ji}$ and $i \neq j$, then it follows that the quadratic form contains $2H_{ij}z_iz_j$. Including $2z_iz_j$ in the feature vector allows for the corresponding parameter to be stored once. A different symmetric-vectorization convention would be equally valid if the feature construction and matrix reconstruction remained mutually consistent.

Saddle-Policy Recovery from the Learned Matrix

For a fixed state, it follows that the input-dependent part of (4.4) is

$$\begin{aligned} Q(x, u_P, u_E) = x^\top H_{xx}x + 2u_P^\top H_{Px}x + 2u_E^\top H_{Ex}x \\ + u_P^\top H_{PP}u_P + 2u_P^\top H_{PE}u_E + u_E^\top H_{EE}u_E. \end{aligned} \tag{4.9}$$

Then, the stationary conditions with respect to the minimizing and maximizing inputs are

$$\frac{\partial Q}{\partial u_P} = 0, \quad \frac{\partial Q}{\partial u_E} = 0. \tag{4.10}$$

Under stationary feedback $u_P = Lx$ and $u_E = Kx$, these conditions give

$$\underbrace{\begin{bmatrix} H_{PP} & H_{PE} \\ H_{EP} & H_{EE} \end{bmatrix}}_{\mathcal{H}_u} \begin{bmatrix} L \\ K \end{bmatrix} = -\underbrace{\begin{bmatrix} H_{Px} \\ H_{Ex} \end{bmatrix}}_{\mathcal{H}_x}. \tag{4.11}$$

Hence, once H has been estimated, the pursuer and evader gains are then recovered from a coupled block solve. The learner does not require a separate actor parameterization since the quadratic action-value matrix supplies an analytical saddle-policy recovery equation.

Moreover, a finite solution of (4.11) is not sufficient. The minimizing block must satisfy

$$H_{PP} \succ 0, \tag{4.12}$$

and the maximizing-player Schur complement must satisfy

$$S_E = H_{EE} - H_{EP}H_{PP}^{-1}H_{PE} \prec 0. \tag{4.13}$$

The first condition provides positive curvature in the pursuer input, while the second provides curvature in the evader direction after the minimizing input has been eliminated. These signs distinguish a local minimax saddle from a stationary point having the wrong game-theoretic ordering.

Additionally, the conditioning of $\mathcal{H}_u$ also matters as a nearly singular input block can produce large gain changes from small coefficient errors even when the least-squares estimate of H appears accurate in Frobenius norm. Thus, the learning problem contains two separate conditioning questions. More specifically, these questions are whether the feature regression reliably identifies θ and whether the recovered input block converts that estimate into reliable, stable saddle gains. The following sections report both since good Bellman fit does not automatically imply robust policy recovery.

4.2 Nominal Q-Learning Policy Iteration

Fixed-Policy Bellman Equation

Q-learning policy iteration begins with a gain pair (L^i, K^i) and evaluates the quadratic action-value function that is generated when those policies are used at the successor state. For each measured transition, define the successor policy actions

$$u_{P,k+1}^{(i)} = L^i x_{k+1}, \quad u_{E,k+1}^{(i)} = K^i x_{k+1}, \tag{4.14}$$

as well as the corresponding successor state-action given by

$$z_{k+1}^{(i)} = \begin{bmatrix} x_{k+1} \\ L^i x_{k+1} \\ K^i x_{k+1} \end{bmatrix}. \tag{4.15}$$

For a fixed policy pair, the undiscounted Bellman identity is

$$Q^i(z_k) = \ell_k + Q^i\left(z_{k+1}^{(i)}\right). \tag{4.16}$$

Hence, substitution of (4.8) into (4.16) yields

$$\left[\phi(z_k) - \phi\left(z_{k+1}^{(i)}\right)\right]^\top \theta^i = \ell_k. \tag{4.17}$$

The equality is exact in the ideal deterministic linear-quadratic setting when the parameterization is exact and the current policy pair generates a finite infinite-horizon value. In finite data with numerical noise, the equations are solved in the least-squares manner.

This interpretation parallels model-based policy evaluation. Recall that in the previous chapter, the Lyapunov equation questions what value is generated by holding (K^i, L^i) fixed. Here, the feature-difference equation questions what quadratic state-action value is consistent with sampled transitions when the same fixed pair is used at the successor state. The analytical plant matrices have been replaced by measured one-step consequences; however, the evaluate-improve logic remains recognizable.

Policy evaluation is only meaningful when the current pair produces a finite value over the sampled operating region. In the nominal linear setting, a Schur stable closed-loop provides the standard infinite-horizon interpretation. More generally, a dataset that wanders outside the region where the quadratic approximation is meaningful can invalidate the interpretation even if the regression is full rank. Data coverage and closed-loop admissibility are thus related but distinct requirements.

Convergence Interpretation and Nominal Verification

A small change in H or in the recovered gains does not by itself prove that the learned Q-function satisfies the Bellman equations. Hence, define the normalized policy evaluation residual as

$$\varepsilon_{B,\mathrm{PI}}^i = \frac{\left\|\Psi^i \widehat{\theta}^i - \ell\right\|_2}{1 + \|\ell\|_2}. \tag{4.18}$$

This quantity tests consistency with the sampled fixed-policy Bellman relation as it should be

interpreted together with the numerical rank and condition number of Ψ^i. A small residual from a rank-deficient regression may reflect nonuniqueness rather than accurate identification, while a full-rank but ill-conditioned regression may fit the batch closely and still yield unstable coefficients under small perturbations.

The reconstructed matrix should also satisfy a symmetry check like

$$\varepsilon^i_{\text{sym}} = \frac{\left\| \hat{H}^i - (\hat{H}^i)^\top \right\|_F}{1 + \left\| \hat{H}^i \right\|_F}. \tag{4.19}$$

When H is formulated directly from independent symmetric coefficients, this quantity is near zero by construction. Reporting it remains useful because it confirms that feature ordering, coefficient placement, and block extraction are mutually consistent.

The final input blocks must satisfy both (4.12) and (4.13), and the joint block $\mathcal{H}_u$ must be sufficiently well-conditioned for the gain solve to be meaningful. The final gain pair is then checked against the closed-loop stability requirement of the common benchmark. This external stability check does not make the learner model-based. Rather, it is a validation step that is performed after learning, just as a model-free estimate can be compared with a known analytical solution without using that solution during the regression.

When a model-based reference $(L_{\text{ref}}, K_{\text{ref}})$ is available, then relative gain errors may be reported as

$$\varepsilon_L = \frac{\|L - L_{\text{ref}}\|_F}{1 + \|L_{\text{ref}}\|_F}, \quad \varepsilon_K = \frac{\|K - K_{\text{ref}}\|_F}{1 + \|K_{\text{ref}}\|_F}. \tag{4.20}$$

These errors test nominal recovery of the same zero-sum solution as they do not enter the learning update and do not establish that Q-learning is better than the Riccati method. Agreement is the expected outcome when both methods are able to solve the same linear-quadratic game and the sampled regression recovers the exact quadratic structure.

4.3 Disturbance-Aware Extension of Q-Learning Policy Iteration

Reasoning for Calibration and Action Labeling Preceding Learning

The nominal policy iteration algorithm assumes that the action stored in each transition is the action that generated the successor state. Persistent command offsets separate the requested and realized inputs:

$$u^{\text{act}}_{P,k} = u^{\text{cmd}}_{P,k} + b_P, \quad u^{\text{act}}_{E,k} = u^{\text{cmd}}_{E,k} + b_E. \tag{4.21}$$

If the dataset stores the commanded values while the plant evolves under the realized values, then both the feature vector and the stage cost are mislabeled. The error is systematic since the same offset appears in every sample, and it cannot be removed by adding more samples of the same incorrectly labeled interaction.

Thus, the model-free extension begins with command-channel calibration. During a calibration interval, the commanded and realized inputs are measured separately as

$$u^{\text{meas}}_{P,k} = u^{\text{cmd}}_{P,k} + b_P + \eta_{P,k}, \quad u^{\text{meas}}_{E,k} = u^{\text{cmd}}_{E,k} + b_E + \eta_{E,k}, \tag{4.22}$$

where $\eta_{P,k}$ and $\eta_{E,k}$ denote input-measurement noise. The sample mean estimates are given by the equations

$$\hat{b}_P = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{P,k}^{\text{meas}} - u_{P,k}^{\text{cmd}} \right), \tag{4.23}$$

$$\hat{b}_E = \frac{1}{N_{\text{cal}}} \sum_{k=1}^{N_{\text{cal}}} \left(u_{E,k}^{\text{meas}} - u_{E,k}^{\text{cmd}} \right). \tag{4.24}$$

No plant-transition matrix appears in these equations. The estimator uses the measured difference at each command channel and thus remains consistent with the model-free information boundary of the Bellman update.

If the measurement noises are zero mean, independent across samples, and have covariances Σ_P and Σ_E, then it follows that

$$\text{Cov}(\hat{b}_P) = \frac{\Sigma_P}{N_{\text{cal}}}, \quad \text{Cov}(\hat{b}_E) = \frac{\Sigma_E}{N_{\text{cal}}}. \tag{4.25}$$

Affine Deployment and Residual Closed-Loop Behavior

After the nominal Q-learning policy iteration converges, then the learned feedback gains are deployed by

$$u_{P,k}^{\text{cmd}} = Lx_k - \hat{b}_P, \quad u_{E,k}^{\text{cmd}} = Kx_k - \hat{b}_E. \tag{4.26}$$

The realized inputs are given by

$$u_{P,k}^{\text{act}} = Lx_k + \tilde{b}_P, \quad u_{E,k}^{\text{act}} = Kx_k + \tilde{b}_E. \tag{4.27}$$

For theoretical interpretation, the resulting plant dynamics are given by

$$x_{k+1} = A_{\text{cl}} x_k + B_P \tilde{b}_P + B_E \tilde{b}_E, \quad A_{\text{cl}} = A + B_P L + B_E K, \tag{4.28}$$

and equation (4.28) is not used by the learner. It is the common physical interpretation of the policy after learning and demonstrates that the affine compensation law replaces the original offsets with their estimation errors.

Let x_k^{nom} and x_k^{comp} denote the nominal and estimated-compensation trajectories from the same initial condition as their deviation satisfies the expression

$$d_{k+1} = A_{\text{cl}} d_k + B_P \tilde{b}_P + B_E \tilde{b}_E, \quad d_0 = 0, \tag{4.29}$$

with the finite-step solution of

$$d_k = \sum_{j=0}^{k-1} A_{\text{cl}}^j \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{4.30}$$

When A_{cl} is Schur stable, then

$$d_\infty = (I - A_{\text{cl}})^{-1} \left(B_P \tilde{b}_P + B_E \tilde{b}_E \right). \tag{4.31}$$

Algorithm 3 Disturbance-Aware Extension of Q-Learning Policy Iteration

Phase I: Calibrate the command channels.
Apply known calibration commands and measure the realized pursuer and evader inputs.
Estimate $\widehat{b}_P$ and $\widehat{b}_E$ from the sample means of command-versus-measurement differences.
Record the calibration spread and state the assumption that the two realized inputs are measured separately.
Phase II: Construct a disturbance-consistent transition batch.
Choose desired realized exploration sequences $v_{P,k}$ and $v_{E,k}$ with sufficient joint excitation.
Send $v_{P,k} - \widehat{b}_P$ and $v_{E,k} - \widehat{b}_E$ to the biased command channels.
Measure and store x_k, $u_{P,k}^{\mathrm{act}}$, $u_{E,k}^{\mathrm{act}}$, x_{k+1}, and the realized stage cost.
Construct the quadratic features from the measured realized inputs and verify feature rank and conditioning.
Phase III: Execute the nominal Q-learning policy iteration.
Apply the nominal Q-learning policy-iteration procedure to the fixed disturbance-consistent batch until the Q-function and gain sequences satisfy the selected convergence and validity checks.
Phase IV: Apply and verify the matched execution cases.
Evaluate nominal, uncompensated, estimated-compensation, and exact-compensation operation with the learned gains.
Report the Bellman diagnostics, feature diagnostics, offset errors, residual affine forcing, analytical recurrence mismatch, and exact-compensation trajectory error.

Recall that the same residual bias relationship appeared in the previous chapter since it is a property of the executed plant, not of the method that is used to obtain L and K.

Exact compensation sets $\hat{b}_P = b_P$ and $\hat{b}_E = b_E$. Then $\tilde{b}_P = \tilde{b}_E = 0$ and (4.28) reduces to the nominal learned-policy dynamics. Thus, exact compensation verifies command bookkeeping independently of Q-function accuracy. A learned policy could be suboptimal and still pass the exact compensation test. Conversely, a highly accurate nominal policy could fail the test if command and realized inputs were handled inconsistently.

4.4 Nominal Q-Learning Value Iteration

Bootstrapped Bellman Target

Q-learning value iteration uses the same measured state-action features, but advances the action-value approximation through a sampled optimality backup. Let θ^i and H^i denote the current Q-function estimate, and let (L^i, K^i) be the saddle gains that are recovered from H^i. For each transition, form the successor policy vector

$$z_{k+1}^{(i)} = \begin{bmatrix} x_{k+1} \\ L^i x_{k+1} \\ K^i x_{k+1} \end{bmatrix}. \tag{4.32}$$

The current bootstrapped target is

$$y_k^i = \ell_k + \phi\left(z_{k+1}^{(i)}\right)^\top \theta^i. \tag{4.33}$$

Let Φ denote the fixed measured feature matrix whose kth row is $\phi(z_k)^\top$. Then, the next parameter estimate satisfies

$$\Phi\theta^{i+1} \approx y^i, \tag{4.34}$$

and is obtained from

$$\widehat{\theta}^{i+1} = \arg\min_{\theta} \|\Phi\theta - y^i\|_2^2. \tag{4.35}$$

After reconstructing a symmetric H^{i+1}, the next gain pair is recovered from its input-related blocks.

The distinction from policy iteration is exact and limited. Policy iteration places the successor-policy feature on the left-hand side and solves for the Q-function of a fixed policy pair given by

$$\left[\phi(z_k) - \phi\left(z_{k+1}^{(i)}\right)\right]^\top \theta^i = \ell_k. \tag{4.36}$$

Value iteration places the current successor Q-value in the target and advances the approximation expressed by

$$\phi(z_k)^\top \theta^{i+1} = \ell_k + \phi\left(z_{k+1}^{(i)}\right)^\top \theta^i. \tag{4.37}$$

The first equation evaluates one policy after improvement, while the second performs one sampled optimality backup using the current value approximation. This is the model-free counterpart of the distinction between the Lyapunov evaluate-improve cycle and the Riccati mapping in the previous chapter.

4.5 Disturbance-Aware Extension of Q-Learning Value Iteration

Algorithm 4 Disturbance-Aware Extension of Q-Learning Value Iteration

Phase I: Calibrate the command channels.
Measure commanded and realized pursuer and evader inputs over the calibration interval.
Compute $\widehat{b}_P$ and $\widehat{b}_E$ from the direct command-mismatch sample means and characterize their measurement uncertainty.
Phase II: Construct the fixed disturbance-consistent dataset.
Convert the desired realized exploration signals into commands by subtracting the offset estimates.
Record the measured realized actions, states, successor states, and realized stage costs.
Construct the fixed feature matrix Φ and verify full numerical rank and acceptable conditioning.

Phase III: Execute the nominal Q-learning value iteration.
Apply the nominal Q-learning value-iteration procedure until the Q-function and gain sequences satisfy the selected convergence and saddle-validity checks.
Phase IV: Apply and verify the matched execution cases.
Evaluate nominal, uncompensated, estimated-compensation, and exact-compensation operation with the learned value-iteration gains.
Report feature and Bellman diagnostics, offset errors, residual affine forcing, analytical recurrence mismatch, and exact-compensation trajectory error.

The persistent command offsets are properties of the measurement and execution channels, not properties of the Bellman arrangement used to estimate H. Thus, Q-learning value iteration uses the same calibration equations, disturbance-consistent action labeling, and affine deployment law developed for policy iteration; only the nominal learning phase changes.

Calibration first estimates the pursuer and evader offsets from direct command-versus-realization differences. The exploration batch then converts the desired realized excitation into commands and records the measured realized inputs in the feature vectors and stage costs. The nominal Q-learning value iteration algorithm applies (4.33)–(4.35) to the fixed batch. After convergence, deployment follows (4.26), and (4.28) describes the residual closed-loop dynamics.

Using the same calibration and data construction method for both model-free algorithms is necessary for a fair comparison. If policy iteration and value iteration used different action measurements, exploration definitions, or offset estimators, then differences in the learned gains could not be attributed solely to the Bellman update. By holding the disturbance-aware layer fixed, the following sections evaluate how the two established Q-learning algorithms recover and verify the nominal saddle solution from the same physically consistent transition data.

4.6 Model-Free Q-Learning Policy Iteration Results

Data Sufficiency and Nominal Policy Recovery

The Q-learning policy iteration algorithm used a fixed set of 500 off-policy transitions; moreover, the augmented state-action vector has dimension 5, so the symmetric Q-function matrix contains 15 unique coefficients. Both the raw and scaled feature matrices had rank 15. The raw feature matrix condition number was approximately 412.79, the scaled condition number was approximately 308.87, and the final policy evaluation condition number was approximately 218.02.

The learned Q-function, gains, and value matrix agreed with the offline model-based reference to

relative errors between approximately 10^{-14} and 10^{-12}. This is consistent with an exact quadratic representation and a sufficiently rich deterministic dataset.

Figure 4.1 demonstrates the rapid decrease in the Q-function and gain changes over five outer-evaluation and improvement steps. The third update moves the iterates into the local convergence region, after which the fourth and fifth updates drive all changes close to numerical precision. The five-step count describes outer policy updates as each update still uses the complete 500 transition dataset.

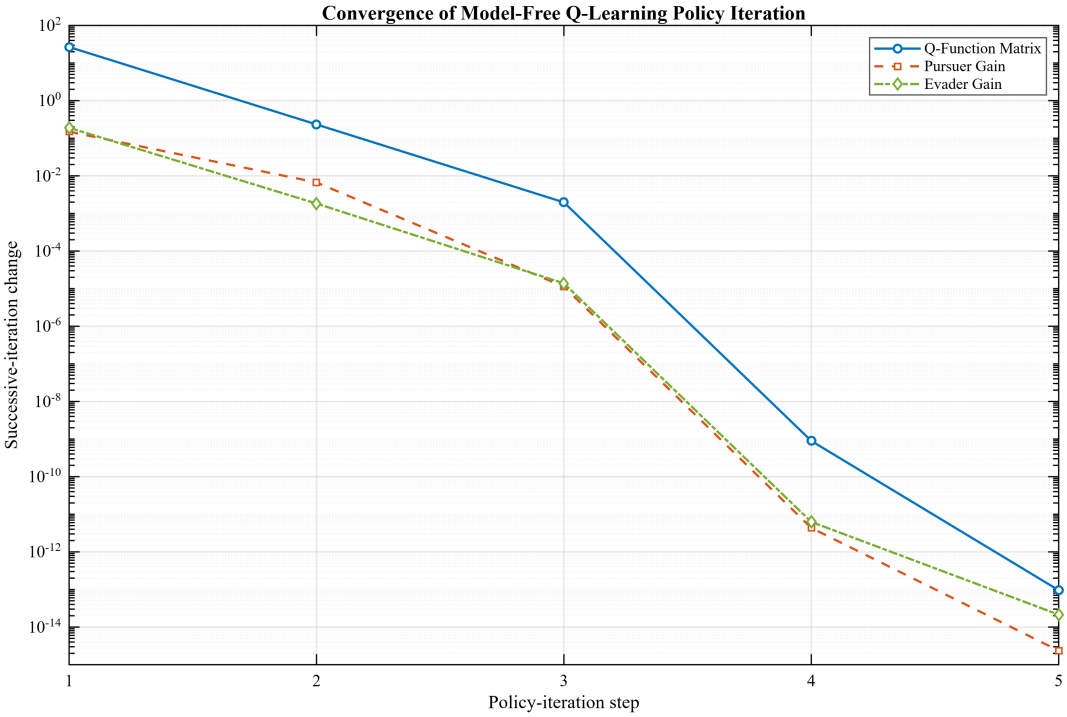


Figure 4.1: Model-Free Q-Learning Policy Iteration Convergence

Direct Command-Mismatch Estimation

The primary estimates were $\widehat{b}_P = 0.0399973010$ and $\widehat{b}_E = -0.0099939136$, with percentage errors of approximately 0.00675% and 0.0609%. Figure 4.2 demonstrates that both channels have nearly identical spread since they use the same realized-input measurement noise standard deviation as well as the same calibration length.

The theoretical densities track the empirical histograms, the Monte Carlo means lie close to the true offsets, and the primary estimates fall within the expected individual estimate intervals. The tighter evader estimate relative to the known-model case is an information structure offset; more specifically, the direct estimator observes each realized channel separately and avoids inversion through the joint plant input map.

Compensation and Cost Recovery

Without compensation, the learned policy produced the same maximum state deviation as the model-based methods, 4.12863×10^{-2}. Estimated compensation reduces this value to 4.16682×10^{-6},

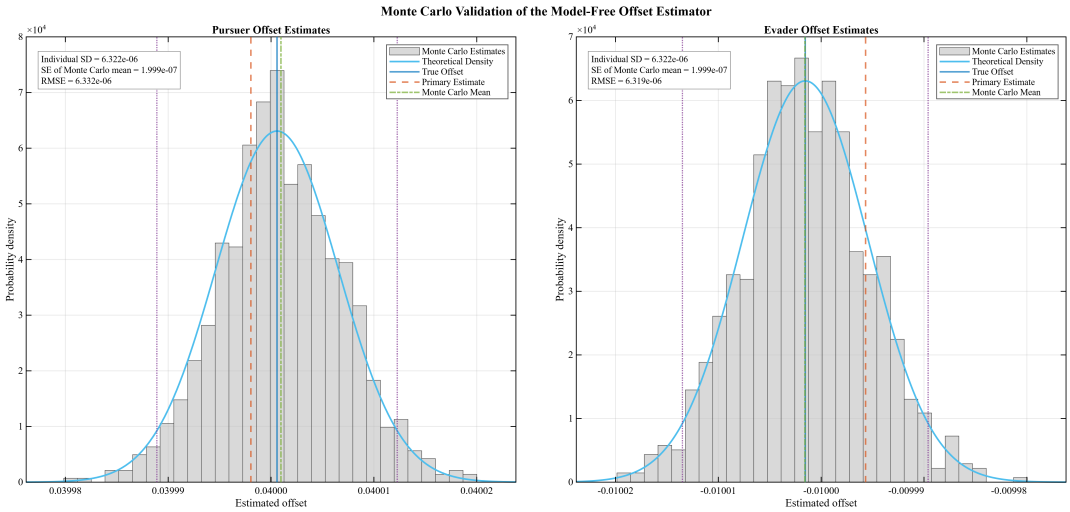


Figure 4.2: Direct Offset-Estimator Monte Carlo Validation for Q-Learning Policy Iteration

corresponding to a reduction of approximately 99.9899%. Exact compensation reproduced the nominal trajectory to approximately 2.78×10^{-17}.

Figure 4.3 demonstrates that the estimated compensation curve increases slightly from an initially zero deviation and then settles at a very small residual affine equilibrium. The effect is not a loss of stability; on the contrary, it is the expected accumulation of a microscale residual offset. The uncompensated curve, by contrast, approaches the same nonzero displacement seen in the model-based cases.

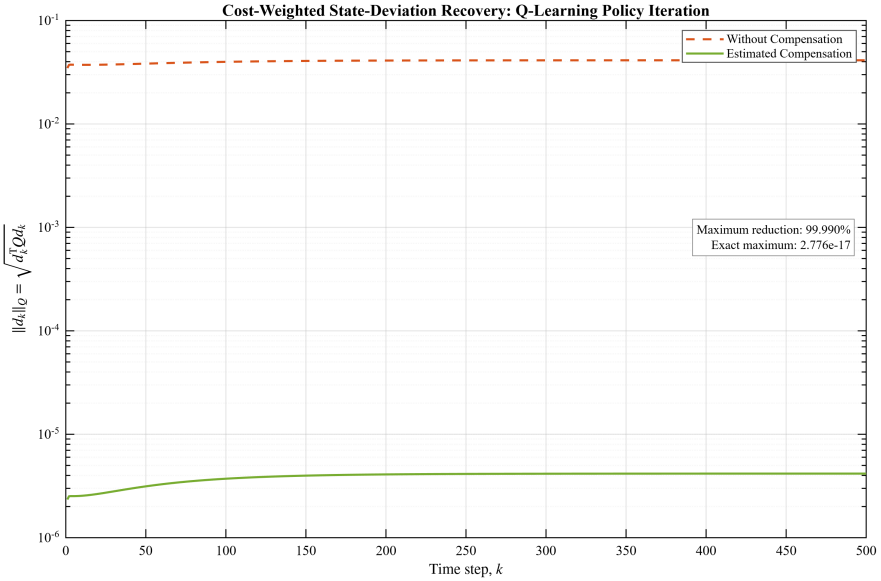


Figure 4.3: Q-Learning Policy Iteration State Deviation Recovery

The analytical deviation recurrence agreed with simulation to approximately 3.74×10^{-15} without compensation and 7.69×10^{-15} with estimated compensation. The signed costs were $J_{\text{nom}} = 3169.85881235989$, $J_{\text{uncomp}} = 3171.20561554776$, $J_{\text{est}} = 3169.85881258328$, and $J_{\text{exact}} = 3169.85881235989$.

Command Channel Behavior

Figure 4.4 separates the policy input, compensated command, and realized input for the direct estimator. The command signals are displaced by the estimated offsets, after which the true channel offsets bring the realized inputs back into near coincidence with the nominal policy inputs. The remaining separation is on the microscale and is determined by the estimator residuals.

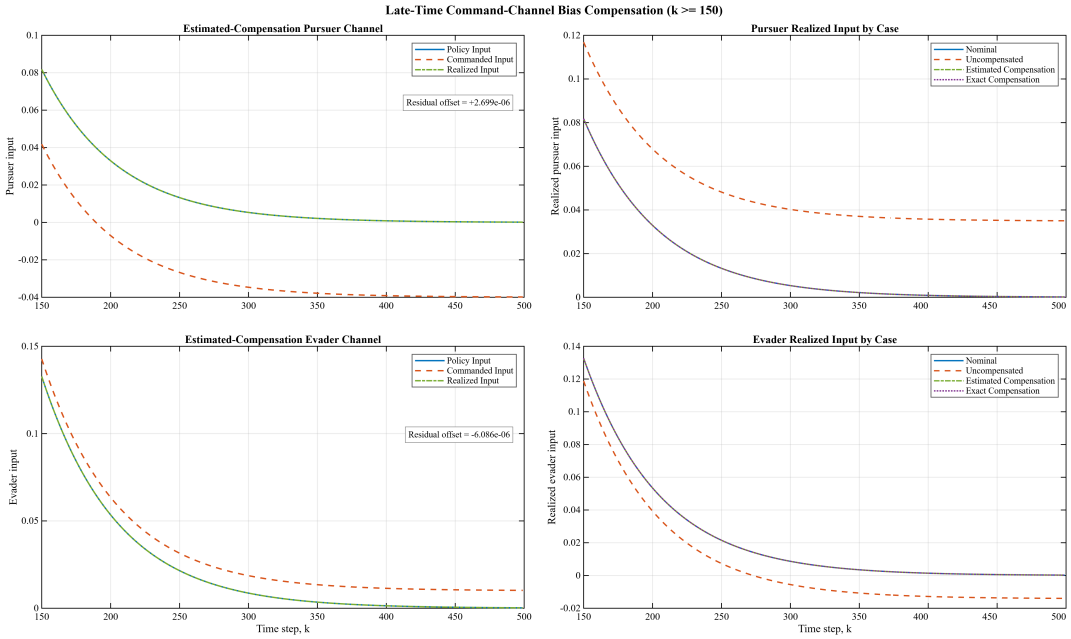


Figure 4.4: Q-Learning Policy Iteration Command Channel Compensation

Therefore, Figures 4.1 through 4.4 demonstrate a complete model-free policy iteration result; moreover, the quadratic policy is recovered from sampled transitions, the direct mismatch estimator identifies both command offsets, and the execution-side correction restores the nominal behavior without inserting the analytical plant matrices into the Bellman regression.

4.7 Model-Free Q-Learning Value Iteration Results

Nominal Learning Convergence

Q-learning value iteration used the same fixed 500 transition dataset and the same 15-parameter quadratic feature representation. The raw and scaled feature matrices again had full rank; moreover, the method converged in 633 value iteration steps, matching the outer step count of the model-based GARE value iteration under the selected tolerance. The learned Q-function, gain pair, and value matrix agreed with the model-based reference errors on the order of 10^{-10}, which is consistent with the stopping tolerance.

Figure 4.5 demonstrates the nearly geometric decrease of the Q-function and gain changes over the main portion of the 633 bootstrapped updates. The late movement in the evader gain curve occurs only near 10^{-12} and is not renewed divergence. In addition, it reflects floating-point sensitivity after the gain has already converged to the displayed precision.

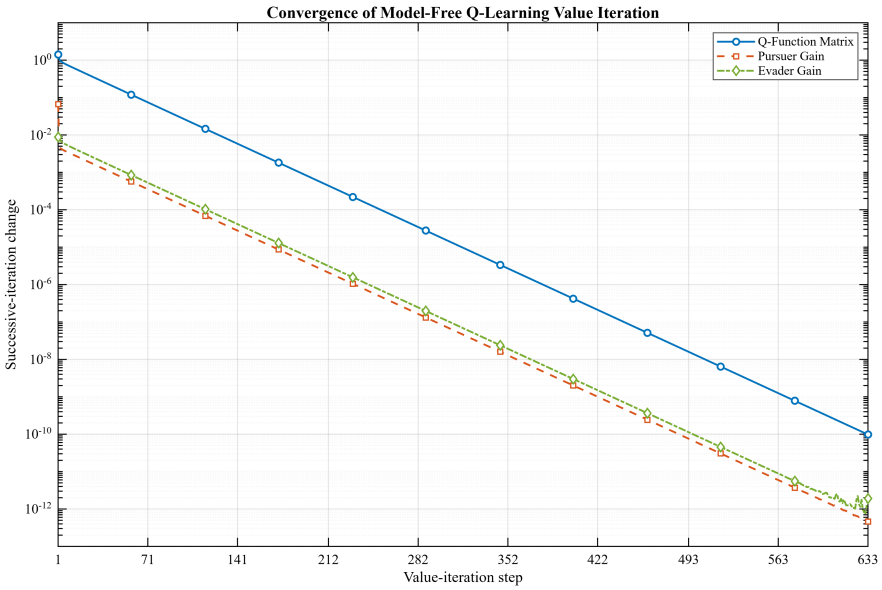


Figure 4.5: Model-Free Q-Learning Value Iteration Convergence

4.8 Matched Four-Method Comparison

Nominal Solution Agreement

Table 4.1 compares the principal numerical results of the four nominal algorithms. All methods produce the same closed-loop spectral radius, game matrix conditioning, and saddle curvatures to the displayed precision. The model-free methods also matched the offline model-based reference with policy iteration reaching relative errors near 10^{-12} or smaller and value iteration reaching errors near 10^{-10}. These differences are consistent with the stopping histories and do not represent meaningfully different controllers in the present benchmark.

Table 4.1: Matched Nominal convergence and Validity Comparison

Method	Outer Steps	Primary Final Change	Defining Residual	$\rho(A_{\mathrm{cl}})$	Game rcond	Curvature Pair
GARE Policy Iteration	4	$\approx 9.0 \times 10^{-10}$	6.40×10^{-15} GARE	0.9819707273	0.56579379	$(1.76184, -0.99851)$
GARE Value Iteration	633	9.44×10^{-11}	9.44×10^{-11} GARE	0.9819707273	0.56579379	$(1.76184, -0.99851)$
Q-Learning Policy Iteration	5	9.54×10^{-14}	4.02×10^{-15} Bellman	0.9819707273	0.56579379	$(1.76184, -0.99851)$
Q-Learning Value Iteration	633	9.79×10^{-11}	3.38×10^{-12} Bellman	0.9819707273	0.56579379	$(1.76184, -0.99851)$

The iteration counts must be interpreted according to algorithm structure; more specifically, each policy iteration update contains a complete fixed-policy evaluation, whereas value iteration advances the value representation through simpler optimality or bootstrapped updates. Therefore, the results establish a difference in outer convergence behavior, not a general wall-clock or sample complexity ranking. Such a ranking would require timed runs, solver-level accounting, repeated trials, and scaling across problem dimensions.

The nominal agreement is itself an important validation result, since all four methods are intended to solve the same linear-quadratic zero-sum game, a substantial disagreement would indicate an inconsistency in signs, action labeling, feature construction, policy recovery, or numerical implementation. Rather, the methods recover the same saddle solution under their respective information structures.

All four nominal controllers are numerically equivalent; moreover, the smaller residual trajectory obtained by the model-free implementation thus reflects the estimator information structure rather than an improved Q-learning controller. The direct estimator observes each realized channel separately, whereas the model-based estimator reconstructs both offsets from their combined state-transition effect through a moderately conditioned input map. Under the specified noise model, direct realized-input measurement produces the more accurate estimate. That is, known-model residual estimation remains applicable when the realized player inputs cannot be measured separately.

Effect of Compensation Across All Methods

The uncompensated maximum deviation is approximately 4.1286×10^{-2} for every method. This common value confirms that the same plant, offsets, initial state, and nominal gain pair are used throughout the comparison. Estimated compensation reduces the deviation by more than 99.7% for the model-based estimator and by more than 99.98% for the direct model-free estimator. Exact compensation reproduces nominal behavior to between approximately 10^{-15} and 10^{-17}, depending on the numerical path used by the program.

The analytical affine-recursion mismatches are on the order of 10^{-15} for all four methods. This check is stronger than visual trajectory overlap since it confirms that the simulated deviations follow the derived residual bias model and are not caused by an undocumented policy change, state reset, or inconsistent action convention.

The signed-cost comparison leads to the same conclusion. The uncompensated case increases the finite-window cost by approximately 1.3468. Estimated compensation returns the cost to within approximately 4.9×10^{-5} for the model-based estimator and 2.3×10^{-7} for the direct estimator. Exact compensation reproduces the nominal cost to the displayed precision. The trajectory and cost metrics thus show that the remaining execution error is governed by the residual offset estimation error.

Chapter 5

Conclusions and Future Work

5.1 Summary

This project report's objective was to investigate disturbance-aware zero-sum pursuit-evasion control through four closely related solution algorithms, which were model-based Game Algebraic Riccati Equation (GARE) policy and value iteration, as well as model-free quadratic Q-learning policy and value iteration. This project report did not intend to replace these nominal algorithms. Rather, it placed them within one mathematical and experimental framework and extended their execution layer to address persistent command-channel offsets affecting the minimizing and maximizing players.

The model-based policy iteration method converged in four outer policy iterations, and the model-free policy iteration method converged in five. Both value iteration methods required 633 value iterations. These counts distinguish the numerical paths, but they do not establish a universal runtime or sample complexity advantage since the work that is performed by a policy iteration update differs from the work performed by a value iteration update. All four methods produced the same nominal closed-loop spectral radius expressed by

$$\rho(A_d + B_P L + B_E K) = 0.9819707273 < 1, \tag{5.1}$$

and the same verified saddle curvatures to the displayed precision. The learned Q-functions and gains also agreed with the model-based reference at the levels implied by their stopping tolerances. The four implementations thus recover the same nominal linear-quadratic saddle solution under the stated numerical conditions.

Without compensation, the common maximum cost-weighted state deviation was

$$\max_k \|d_k\|_Q \approx 4.1286 \times 10^{-2}. \tag{5.2}$$

Known-model transition residual estimation reduced the maximum deviation to approximately 1.1350×10^{-4}, or around 99.725%. Direct command mismatch estimation reduced it to approximately 4.1668×10^{-6}, or around 99.990%. The smaller model-free residual does not indicate a better nominal Q-learning controller. The nominal controllers are numerically equivalent; moreover, the difference arises since the direct estimator observes each realized channel separately, whereas the model-based estimator reconstructs both offsets from their combined state-transition effect through $\begin{bmatrix} B_P & B_E \end{bmatrix}$.

The signed cost results support the same interpretation; that is, the uncompensated offset increased the finite-window cost by approximately 1.3468 relative to nominal. Estimated compensation returned the model-based cases to within approximately 4.9×10^{-5} of the nominal cost and the model-free cases to within approximately 2.3×10^{-7}. Exact compensation reproduced the nominal trajectory and signed cost to the reported precision. With the nominal gain pair fixed, the disturbed state deviation followed

$$\begin{aligned} d_{k+1} = &(A_d + B_P L + B_E K)\, d_k \\ &+ B_P \left(b_P - \widehat{b}_P\right) + B_E \left(b_E - \widehat{b}_E\right). \end{aligned} \tag{5.3}$$

The simulated and independently propagated deviations agreed to approximately 10^{-15} in all four programs. The compensated trajectories thus approach nominal behavior since the estimated offsets reduce the affine forcing by the residual estimation errors predicted by the model.

5.2 Further Research

Nonlinear Vehicle and Platform Validation

The most direct continuation of this project report is to replace the reduced-order benchmark with a nonlinear two-vehicle model. A kinematic bicycle formulation would depict each agent through planar position, heading, speed, wheelbase, acceleration, and steering, while pursuit-evasion would still give the local game interpretation, nominal linearization, estimator organization, compensation cases, and validation logic.

As a result, this extension should proceed first in MATLAB or QLabs while maintaining the current benchmark as a reference. Tests across initial geometries, speeds, wheelbase values, and capture radii would demonstrate where a local saddle policy remains useful and where repeated linearization, gain scheduling, or nonlinear dynamic programming becomes necessary. Then, QLabs should be used to verify command mapping, state extraction, timing, steering calibration, and multi-vehicle coordination before physical QCar deployment.

A QCar-style implementation would also need vector-valued command offsets since each vehicle can have separate longitudinal and steering commands. Thus, the scalar offsets can be generalized as follows

$$b_P = \begin{bmatrix} b_{P,a} & b_{P,\delta} \end{bmatrix}^\top, \quad b_E = \begin{bmatrix} b_{E,a} & b_{E,\delta} \end{bmatrix}^\top, \tag{5.4}$$

where the components denote acceleration or throttle bias and steering bias. The resulting identifiability problem would rely on operating point, input-direction independence, and the different noise and bandwidth characteristics of the longitudinal and steering commands.

Physical QCar experiments should follow only after the QLabs interface is verified; moreover, hardware testing would require synchronized state estimation, repeatable reset procedures, bounded commands, emergency stopping, workspace constraints, and clear separation between calibration and evaluation data. The initial objective should be to validate the command-channel model as well as to determine whether persistent offsets are able to be measured, whether affine compensation reduces execution error, and whether nominal and exact compensation bookkeeping behaves as predicted.

Estimation, Sensing, and Execution Constraints

The current calibration is completed before the compensated execution run. Online estimation could instead use recursive least-squares, a Kalman filter bias state, an unknown-input observer, or a moving-window estimator to update $\widehat{b}_P$ and $\widehat{b}_E$ while the game is in progress. As a result, the analysis would then need to account for estimator transients, interaction between policy and excitation, slowly varying drift, and abrupt faults.

Output-feedback implementation would extend the framework beyond exact state measurement. A practical vehicle platform may provide noisy position, orientation, encoder, inertial, and camera measurements. Thus, state estimation and input bias estimation would become coupled, motivating either an augmented state-and-input observer or an output-feedback Q-learning formulation in which the controller operates on estimated states.

Actuator saturation, rate limits, dead-zones, and command relay need separate treatment since they cannot be represented globally as constant offsets. Therefore, candidate approaches include saturation-aware policy iteration, constrained zero-sum model predictive control, reference governor or safety-filter layers, and delay-augmented dynamic games. The command bookkeeping developed in this project report remains applicable since each approach needs to distinguish policy output, command input, and realized input.

These directions extend a completed foundation; moreover, this research fixes the nominal game, establishes model-based and model-free solution paths, defines the persistent command offset problem, derives the compensation effect, and then verifies the result in MATLAB. Therefore, nonlinear vehicles, online estimation, constraints, QLabs, hardware, and multi-agent systems are the next research directions rather than missing components of the current contribution.

In conclusion, the disturbance-aware execution is able to be added consistently to both model-based and structured model-free zero-sum control without obscuring the nominal game solution. When the offsets are identifiable or directly measurable, affine compensation then recovers the intended saddle policy behavior to the level set by estimator accuracy. The four algorithms differ in information and computation path; however, they concur on the nominal game as well as on the mathematical effect of residual bias. As a result, that agreement provides the theoretical and numerical foundation that will be required for future pursuit-evasion implementation on more realistic autonomous vehicle models and platforms.

References

[1] Tamer Basar and Geert Jan Olsder. *Dynamic Noncooperative Game Theory*. 2nd ed. Philadelphia, PA: SIAM, 1999.

[2] Isaac E. Weintraub, Meir Pachter, and Eloy Garcia. "An Introduction to Pursuit-Evasion Differential Games". In: *2020 American Control Conference (ACC)*. Denver, CO, USA, 2020. DOI: 10.23919/ACC45564.2020.9147205.

[3] Meir Pachter and K. D. Pham. "Discrete-Time Linear-Quadratic Dynamic Games". In: *Journal of Optimization Theory and Applications* 146.1 (2010), pp. 151–179. DOI: 10.1007/s10957-010-9661-x.

[4] Asma Al-Tamimi, Frank L. Lewis, and Murad Abu-Khalaf. "Model-Free Q-Learning Designs for Linear Discrete-Time Zero-Sum Games With Application to H-Infinity Control". In: *Automatica* 43.3 (2007), pp. 473–481. DOI: 10.1016/j.automatica.2006.09.019.

[5] Syed Ali Asad Rizvi and Zongli Lin. *Output Feedback Reinforcement Learning Control for Linear Systems*. Cham, Switzerland: Birkhäuser, 2023. DOI: 10.1007/978-3-031-15858-2.

[6] Biao Luo, Yin Yang, and Derong Liu. "Policy Iteration Q-Learning for Data-Based Two-Player Zero-Sum Game of Linear Discrete-Time Systems". In: *IEEE Transactions on Cybernetics* 51.7 (2021), pp. 3630–3640. DOI: 10.1109/TCYB.2020.2970969.

[7] Jianguo Zhao et al. "Novel Single-Loop Policy Iteration for Linear Zero-Sum Games". In: *Automatica* 163 (2024), p. 111551. DOI: 10.1016/j.automatica.2024.111551.

[8] Liangyuan Guo, Bing-Chang Wang, and Ji-Feng Zhang. "On-Policy and Off-Policy Value Iteration Algorithms for Stochastic Zero-Sum Dynamic Games". In: *Journal of Systems Science and Complexity* 38.1 (2025), pp. 421–435. DOI: 10.1007/s11424-025-4572-y.

[9] Jiduan Wu et al. "Learning Zero-Sum Linear Quadratic Games with Improved Sample Complexity and Last-Iterate Convergence". In: *SIAM Journal on Control and Optimization* 63.5 (2025), pp. 3244–3271. DOI: 10.1137/23M1626104.

[10] Kun Yang et al. "A Review of Reinforcement Learning Approaches for Pursuit-Evasion Games". In: *Chinese Journal of Aeronautics* 39.6 (2026), p. 103940. DOI: 10.1016/j.cja.2025.103940.

[11] Zewei Wang et al. *Reinforcement Learning in Pursuit-Evasion Differential Game: Safety, Stability and Robustness*. arXiv preprint. 2025. DOI: 10.48550/arXiv.2507.19516. arXiv: 2507.19516 [eess.SY]. URL: https://arxiv.org/abs/2507.19516.

[12] Burak M. Gonultas and Volkan Isler. *Learning to Play Pursuit-Evasion with Dynamic and Sensor Constraints*. arXiv preprint. 2024. DOI: 10.48550/arXiv.2405.05372. arXiv: 2405.05372 [cs.RO]. URL: https://arxiv.org/abs/2405.05372.

[13] Rufus Isaacs. *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. New York, NY: John Wiley & Sons, 1965.

[14] Yu-Chi Ho. "A Note on Linear-Quadratic Pursuit-Evasion Differential Games". In: *Journal of Optimization Theory and Applications* 5.6 (1970), pp. 449–451. DOI: 10.1007/BF00927443.

[15] Arunabha Bagchi and Geert Jan Olsder. "Linear Quadratic Stochastic Pursuit-Evasion Games". In: *Applied Mathematics and Optimization* 7 (1981), pp. 95–123. DOI: 10.1007/BF01442109.

[16] Josef Shinar. "Solvability of Linear-Quadratic Differential Games Associated with Pursuit-Evasion Problems". In: *International Game Theory Review* 10.4 (2008), pp. 619–636. DOI: 10.1142/S0219198908002060.

[17] Jingrui Sun and Jiongmin Yong. "Linear Quadratic Stochastic Differential Games: Open-Loop and Closed-Loop Saddle Points". In: *SIAM Journal on Control and Optimization* 52.6 (2014), pp. 4082–4121. DOI: 10.1137/140953642.

[18] Jingrui Sun. "Two-Person Zero-Sum Stochastic Linear-Quadratic Differential Games". In: *SIAM Journal on Control and Optimization* 59.3 (2021), pp. 1804–1829. DOI: 10.1137/20M1340368.

[19] Andrea Monti et al. "Feedback and Open-Loop Nash Equilibria for Linear Quadratic Infinite-Horizon Discrete-Time Dynamic Games". In: *SIAM Journal on Control and Optimization* (2024). DOI: 10.1137/23M1579960.

[20] Jingrui Sun and Jiongmin Yong. "Long-Time Behavior of Zero-Sum Linear-Quadratic Stochastic Differential Games". In: *SIAM Journal on Control and Optimization* 63.6 (2025), pp. 3961–3989. DOI: 10.1137/24M1664897.

[21] David L. Kleinman. "On an Iterative Technique for Riccati Equation Computations". In: *IEEE Transactions on Automatic Control* 13.1 (1968), pp. 114–115. DOI: 10.1109/TAC.1968.1098829.

[22] Gary A. Hewer. "An Iterative Technique for the Computation of the Steady-State Gains for the Discrete Optimal Regulator". In: *IEEE Transactions on Automatic Control* 16.4 (1971), pp. 382–384. DOI: 10.1109/TAC.1971.1099755.

[23] E. F. Mageirou. "Iterative Techniques for Riccati Game Equations". In: *Journal of Optimization Theory and Applications* (1977). DOI: 10.1007/BF00936720.

[24] Murad Abu-Khalaf, Frank L. Lewis, and Jie Huang. "Policy Iterations on the Hamilton–Jacobi–Isaacs Equation for H-Infinity State-Feedback Control with Input Saturation". In: *IEEE Transactions on Automatic Control* 53.6 (2008), pp. 1483–1488. DOI: 10.1109/TAC.2007.914960.

[25] Huaguang Wu and Biao Luo. "Simultaneous Policy Update Algorithms for Learning the Solution of Linear Continuous-Time H-Infinity State-Feedback Control". In: *Information Sciences* (2013). DOI: 10.1016/j.ins.2012.08.031.

[26] Jie Li et al. "Relaxed Policy Iteration Algorithm for Nonlinear Zero-Sum Games with Application to H-Infinity Control". In: *IEEE Transactions on Automatic Control* 69.1 (2024), pp. 426–433. DOI: 10.1109/TAC.2023.3266277.

[27] Benita Nortmann et al. “Nash Equilibria for Linear Quadratic Discrete-Time Dynamic Games via Iterative and Data-Driven Algorithms”. In: *IEEE Transactions on Automatic Control* 69.10 (2024), pp. 6561–6575. DOI: 10.1109/TAC.2024.3375249.

[28] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control, Volume I*. 4th ed. Belmont, MA: Athena Scientific, 2017.

[29] Christopher J. C. H. Watkins and Peter Dayan. “Q-Learning”. In: *Machine Learning* 8.3–4 (1992), pp. 279–292. DOI: 10.1007/BF00992698.

[30] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press, 2018.

[31] Steven J. Bradtke and Andrew G. Barto. “Linear Least-Squares algorithms for temporal difference learning”. In: *Machine Learning* 22 (1996), pp. 33–57. DOI: 10.1007/bf00114723.

[32] Michail G. Lagoudakis and Ronald Parr. “Least-Squares Policy Iteration”. In: *Journal of Machine Learning Research* 4 (2003), pp. 1107–1149.

[33] Asma Al-Tamimi, Murad Abu-Khalaf, and Frank L. Lewis. “Adaptive Critic Designs for Discrete-Time Zero-Sum Games With Application to H-Infinity Control”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B* 37.1 (2007), pp. 240–247. DOI: 10.1109/TSMCB.2006.880135.

[34] Frank L. Lewis and Draguna Vrabie. “Reinforcement Learning and Adaptive Dynamic Programming for Feedback Control”. In: *IEEE Circuits and Systems Magazine* 9.3 (2009), pp. 32–50. DOI: 10.1109/MCAS.2009.933854.

[35] M. Liu et al. “Finite-horizon Q-learning for discrete-time zero-sum games with application to H_∞ control”. In: *Asian Journal of Control* 25.4 (2023), pp. 3160–3172.

[36] Mingxiang Liu et al. “Output Feedback Q-Learning for Discrete-Time Finite-Horizon Zero-Sum Games with Application to H-Infinity Control”. In: *Neurocomputing* 529 (2023), pp. 48–55. DOI: 10.1016/j.neucom.2023.01.050.

[37] Bosen Lian et al. “Off-Policy Inverse Q-Learning for Discrete-Time Antagonistic Unknown Systems”. In: *Automatica* 155 (2023), p. 111171. DOI: 10.1016/j.automatica.2023.111171.

[38] Liangyuan Guo, Bing-Chang Wang, and Hailing Dong. “Q-Learning Design for Discrete-Time Stochastic Zero-Sum Games”. In: *2023 China Automation Congress*. 2024, pp. 6423–6426. DOI: 10.1109/CAC59555.2023.10451449.

[39] Yuan Wang et al. “Neural Q-Learning for Discrete-Time Nonlinear Zero-Sum Games with Adjustable Convergence Rate”. In: *Neural Networks* 175 (2024), p. 106274. DOI: 10.1016/j.neunet.2024.106274.

[40] Yun Wang et al. “Zero-Sum Game-Based Optimal Control for Discrete-Time Markov Jump Systems: A Parallel Off-Policy Q-Learning Method”. In: *Applied Mathematics and Computation* 467 (2024), p. 128462. DOI: 10.1016/j.amc.2023.128462.

[41] Z. Jiang et al. “Data-Based Discrete-Time Two-Player Zero-Sum Delayed Games via Q-Learning”. In: *Neurocomputing* (2025). URL: https://www.sciencedirect.com/science/article/pii/S0925231225003819.

[42] Shihan Liu, Zhi Chen, and Dongxu Gao. “Safe Reinforcement Learning for Discrete-Time Nonlinear Zero-Sum Games with Unknown State Constraints and Asymmetric Input Constraints”. In: *Neurocomputing* 651 (2025), p. 131026. DOI: 10.1016/j.neucom.2025.131026.

[43] Kaiqing Zhang, Zhuoran Yang, and Tamer Basar. *Policy Optimization Provably Converges to Nash Equilibria in Zero-Sum Linear Quadratic Games*. 2019. arXiv: 1906.00729 [math.OC].

[44] Kenneth R. Muske and Thomas A. Badgwell. "Disturbance Modeling for Offset-Free Linear Model Predictive Control". In: *Journal of Process Control* 12.5 (2002), pp. 617–632. DOI: 10.1016/S0959-1524(01)00051-8.

[45] Gabriele Pannocchia and James B. Rawlings. "Disturbance Models for Offset-Free Model-Predictive Control". In: *AIChE Journal* 49.2 (2003), pp. 426–437. DOI: 10.1002/aic.690490213.

[46] Urban Maeder, Francesco Borrelli, and Manfred Morari. "Linear Offset-Free Model Predictive Control". In: *Automatica* 45.10 (2009), pp. 2214–2222. DOI: 10.1016/j.automatica.2009.06.005.

[47] Isah A. Jimoh et al. "Offset-Free Model Predictive Control: A Study of Different Formulations with Further Results". In: *2020 28th Mediterranean Conference on Control and Automation*. 2020, pp. 671–676. DOI: 10.1109/MED48518.2020.9183056.

[48] Steven Gillijns and Bart De Moor. "Unbiased Minimum-Variance Input and State Estimation for Linear Discrete-Time Systems". In: *Automatica* (2007). DOI: 10.1016/j.automatica.2006.08.002.

[49] Sze Zheng Yong, Minghui Zhu, and Emilio Frazzoli. "A Unified Filter for Simultaneous Input and State Estimation of Linear Discrete-Time Stochastic Systems". In: *Automatica* 63 (2016), pp. 321–329. DOI: 10.1016/j.automatica.2015.10.040.

[50] Julian Berberich et al. "Linear Tracking MPC for Nonlinear Systems—Part II: The Data-Driven Case". In: *IEEE Transactions on Automatic Control* 67.9 (2022), pp. 4406–4421. DOI: 10.1109/TAC.2022.3166851.

[51] Steven J. Kuntz and James B. Rawlings. "Maximum Likelihood Identification of Linear Models with Integrating Disturbances for Offset-Free Control". In: *IEEE Transactions on Automatic Control* 70.9 (2025), pp. 5675–5689. DOI: 10.1109/TAC.2025.3547607.

[52] Steven J. Kuntz and James B. Rawlings. "Offset-Free Model Predictive Control: Stability Under Plant-Model Mismatch". In: *IEEE Transactions on Automatic Control* 71.5 (2026), pp. 3136–3151. DOI: 10.1109/TAC.2025.3637102.

[53] Baibin Yang et al. "ℓ_1-ATSXKF-Based State and Bias Estimation for Nonlinear Systems with Non-Gaussian Process Noise". In: *IET Control Theory & Applications* (2025). DOI: 10.1049/cth2.12769.